

(nano) PROMELA

גרא וייס
המחלקה למדעי המחשב
אוניברסיטת בן-גוריון





שפות המידול

מערכת מעברים

גרף תוכנית

שזירת
מערכות
מעברים

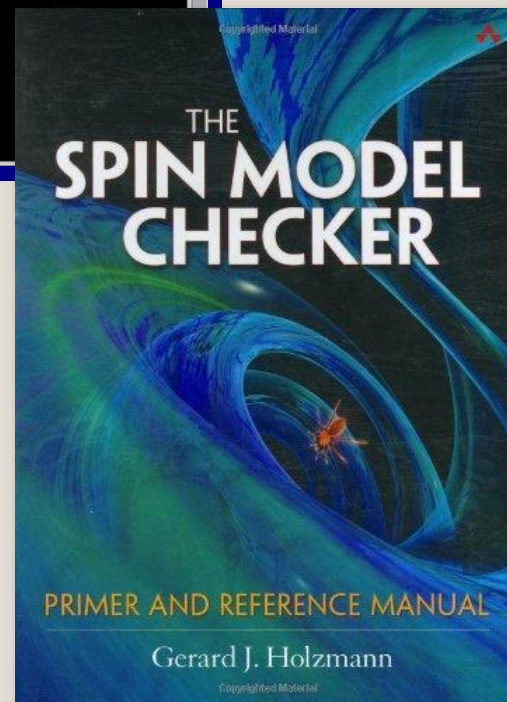
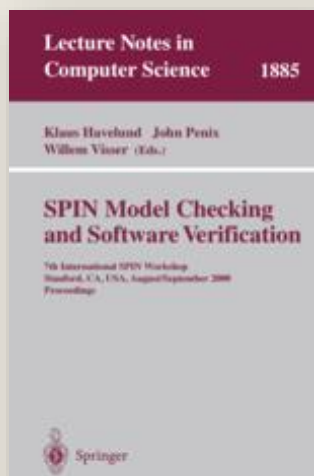
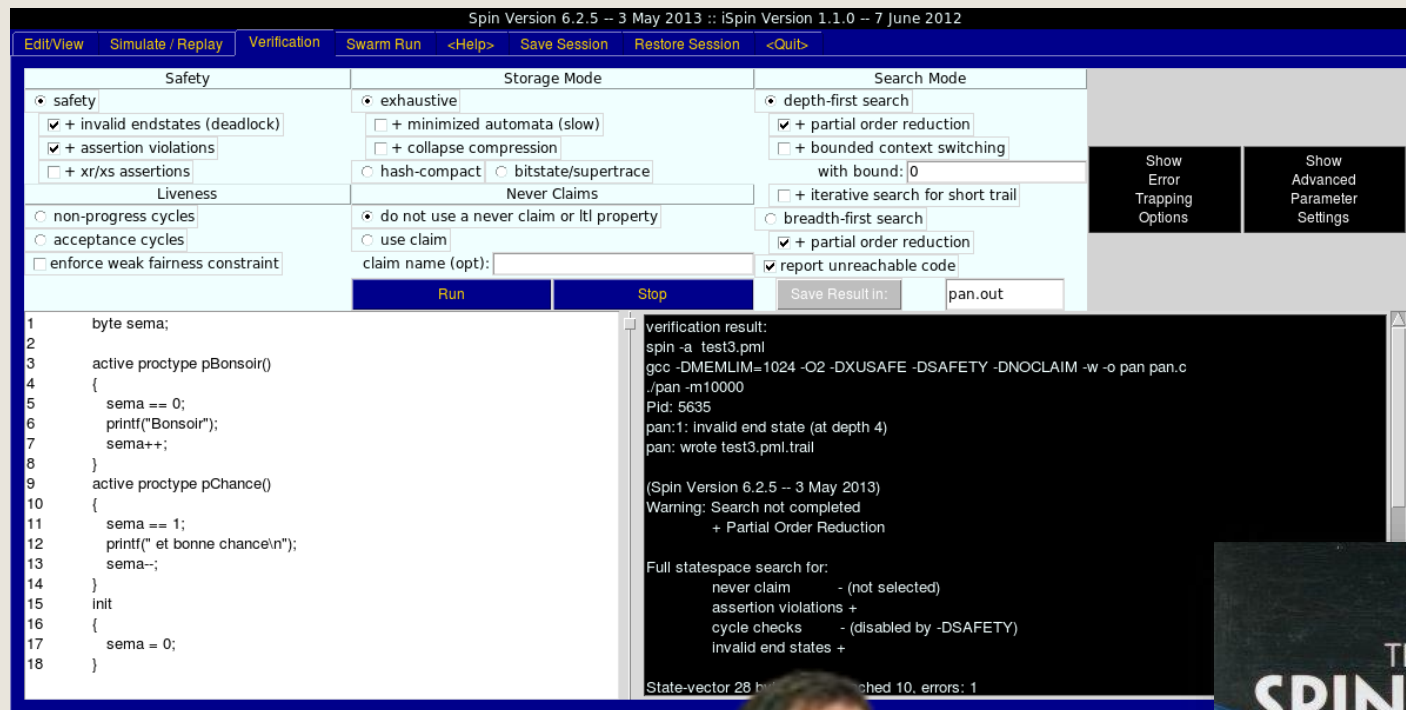
מעגל
חומרה

מערכת
ערוצים

שזירה עם
תקשורת
סנכרונית

שזירה בלי
תקשורת

Promela



nanoPromela



אנחנו נדבר על **גרסה מוקטנת** שנקרא לה nano PROMELA



- משתנים משותפים ותקשורת מבוססת ערוצים

- הסמנטיקה הפורמאלית מבוססת על מערכות ערוצים

- תהליכים מוגדרים באמצעות שפת פקודות מותנות

- אין הגדרה מפורשת של הפעולות. מוגדרות באמצעות הפקודות.



דוגמה: מכונת מכירת משקאות

do

:: true -> skip; /* Coin Insertion */

if :: ns > 0 -> ns:=ns-1

:: nc > 0 -> nc:=nc-1

:: nc==0 && ns==0 -> skip

fi

:: true -> atomic {nc:=10; ns:=10}

od

בחירות לא
דטרמיניסטיות

דוגמה: האלגוריתם של פטרסון

קוד עבור תהליך P_1 :

```
do :: true ->
  /* Noncritical Section */
  atomic { b1:=true; x:=2 };
  if ::(x==1) ∨ ¬b2 -> crit1:=true fi;
  /* Critical Section */
  atomic { crit1:=false; b1:=false }
od
```

if "נתקע" אם
אף אחד
מהתנאים לא
מתקיים

do ממשיך הלאה אם אף אחד מהתנאים לא מתקיים
וחוזר להתחלה לאחר ביצוע צעד

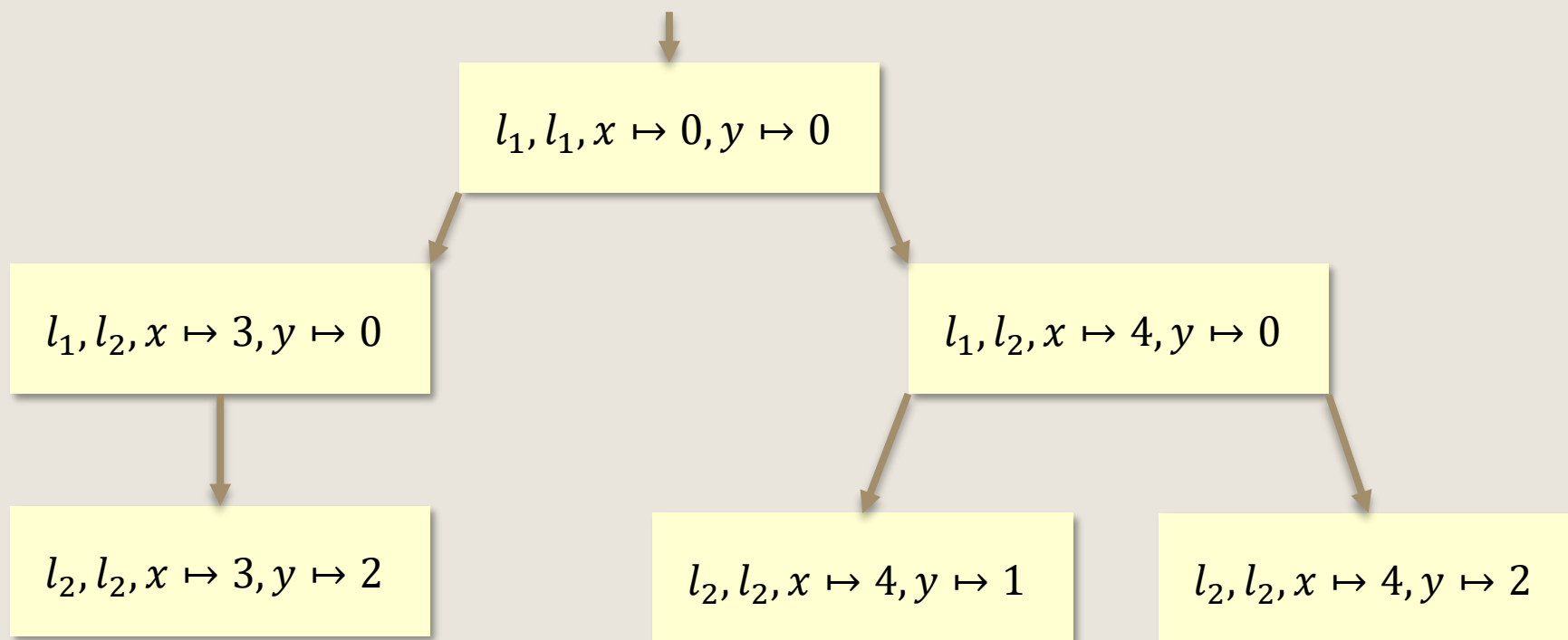
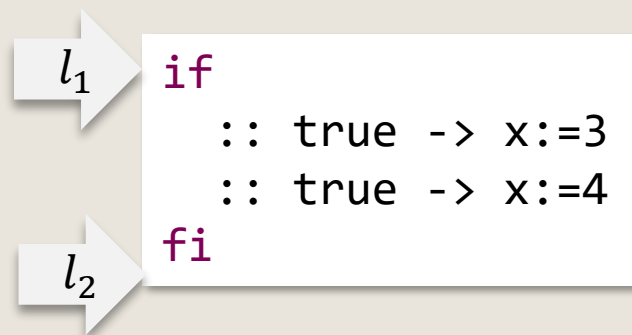
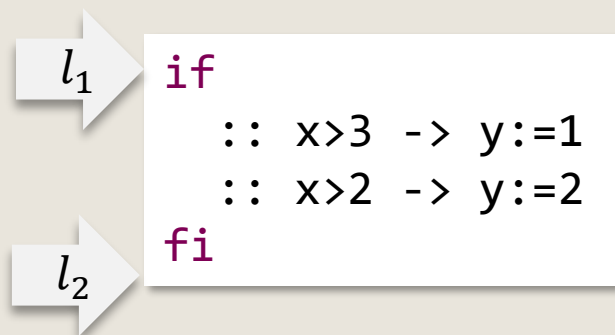


פקודות תנאי

$$\text{if } ::g_1 \rightarrow stmt_1 \ \cdots \ ::g_n \rightarrow stmt_n \ \text{fi}$$

- בחירה אי דטרמיניסטית בין פקודות $stmt_i$ שהתנאים g_i שלהן מתקיימים
- סמנטיקה של **test-and-set**: בדיקת התנאי + בחירת הפקודה + ביצוע הצעד הראשון של הפקודה מתבצעים באופן אטומי.
- פקודת if-fi נתקעת אם אף אחד מהתנאים אינו מתקיים
- תהליכים מקבילים יכולים לשחרר את החסימה ע"י שינוי משתנים משותפים
- למשל: כאשר $y = 0$, הפקודה **if ::y>0 -> x := 39 fi** ממתינה ש y יהיה גדול מאפס
- **הכללה** של שפות תכנות רגילות:
 - **if ::g→stmt₁ ::¬g→stmt₂ fi** שקול ל- **if g then stmt₁ else stmt₂ fi**
 - **if ::g→stmt₁ ::¬g→skip fi** שקול ל- **if g then stmt₁ fi**

הדגמת הסמנטיקה של פקודות תנאי



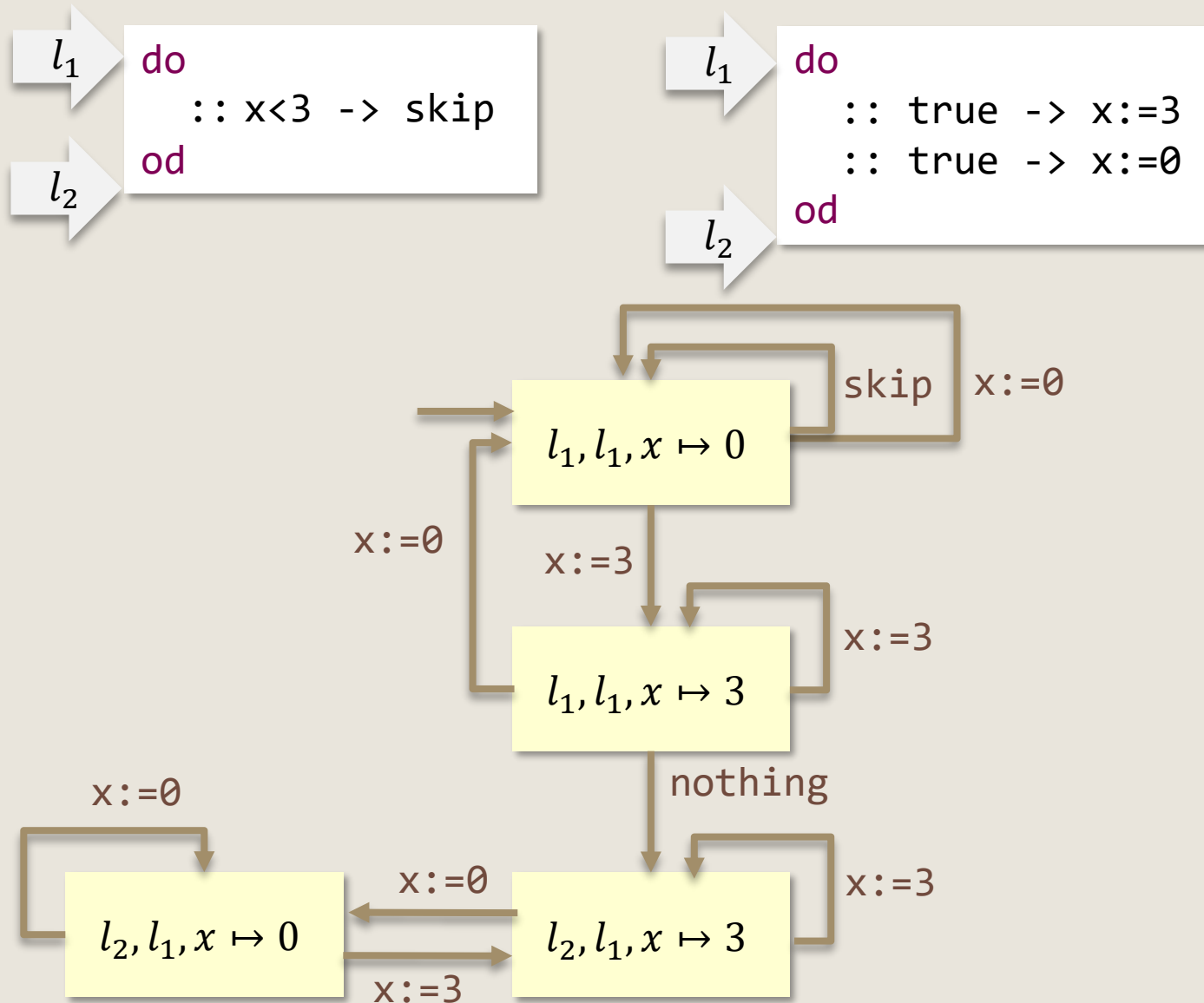


פקודות לולאה

$$\text{do} :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n \text{ od}$$

- ביצוע חוזר של כל הפקודות $stmt_i$ שתנאיהן g_i מתקיימים
- אין תקיעה כשכל התנאים מופְּרִים - הלולאה מסתיימת וממשיכים בביצוע התכנית
- הכללה של שפות תכנות רגילות:
- $\text{do} :: g \rightarrow stmt \text{ od}$ שקול ל- $\text{while } g \text{ do } stmt \text{ od}$
- אין פקודת break ליציאה מלולאות

הדגמת הסמנטיקה של פקודות לולאה



אימות אלגוריתם למניעה הדדית



```
boolean array b(0;1) integer k,  
process i, with i either 0 or 1;  
C0:    b(i) := false;  
C1:    if k != i then begin  
C2:    if not (b(1-i)) then go to C2;  
        else k := i; go to C1 end;  
        else critical section;  
        b(i) := true;  
        remainder of program;  
        go to C0;
```



פרוטוקול שהוצע למניעה הדדית

"Comments on a Problem in Concurrent
Programming Control," by H. Hyman
Communications of the ACM, 1966, Volume 9

```
int cnt, k, b[2];  
  
active [2] proctype P() {  
    assert(_pid == 0 || _pid == 1);  
    C0: b[_pid] = 0;  
    C1: if  
        :: (k != _pid) ->  
            (b[1 - _pid]);  
            k = _pid;  
            goto C1  
  
        :: else ->  
            cnt = cnt+1;  
            assert(cnt == 1); /* crit. sect. */  
            cnt = cnt-1;  
            b[_pid] = 1;  
    fi;  
    goto C0  
}
```



תרגום הפרוטוקול ל-PROMELA

גילוי אוטומטי של טעות בפרוטוקול



```

1: proc 1 (P) line 5 "hy66" (state 1) [assert((( _pid==0) || (_pid==1)))]
2: proc 1 (P) line 7 "hy66" (state 2) [b[_pid] = 0]
3: proc 1 (P) line 9 "hy66" (state 3) [(k!=_pid)]
4: proc 0 (P) line 5 "hy66" (state 1) [assert((( _pid==0) || (_pid==1)))]
5: proc 0 (P) line 7
6: proc 0 (P) line 14
7: proc 0 (P) line 15
8: proc 0 (P) line 16
9: proc 0 (P) line 17
10: proc 0 (P) line 19
11: proc 1 (P) line 10
12: proc 0 (P) line 7
13: proc 0 (P) line 14

14: proc 1 (P) line 11 "hy66" (state 5) [k = _pid]
15: proc 1 (P) line 14 "hy66" (state 7) [else]
16: proc 1 (P) line 15 "hy66" (state 8) [cnt = (cnt+1)]
17: proc 1 (P) line 16 "hy66" (state 9) [assert((cnt==1))]
18: proc 0 (P) line 15 "hy66" (state 8) [cnt = (cnt+1)]
spin: line 16 "hy66", Error: assertion violated
19: proc 0 (P) line 16 "hy66" (state 9) [assert((cnt==1))]
spin: trail ends after 19 steps
#processes: 2
    cnt = 2
    k = 1
    b[0] = 0
    b[1] = 0
19: proc 1 (P) line 17 "hy66" (state 10)
19: proc 0 (P) line 17 "hy66" (state 10)
2 processes created

```

חידה: גשר צר מאוד



- ארבעה חיילים צריכים לחצות גשר
- מהירויות החיילים: 5,10,20,25 דקות בהתאמה
- **לכל היותר**, שניים יכולים להיות על הגשר בו זמנית
- יש לפיד אחד 
- מדובר בשעת לילה, הגשר צר מאוד, **חייבים ללכת עם לפיד**
- האם אפשר להעביר את כולם בתוך 60 דקות?



דוגמה: גשר צר מאוד



```

byte now=0; byte time[4]; bit where[4]; bit torch; byte i,j;
active proctype bridge() {
    time[0] = 5; time[1] = 10; time[2] = 20; time[3] = 25;
    do :: now <= 60 ->
        assert(!(where[0]==1 && where[1]==1 && where[2]==1 && where[3]==1));
        if :: where[0] == torch -> i = 0
            :: where[1] == torch -> i = 1
            :: where[2] == torch -> i = 2
            :: where[3] == torch -> i = 3
        fi;
        if :: where[0] == torch -> j = 0
            :: where[1] == torch -> j = 1
            :: where[2] == torch -> j = 2
            :: where[3] == torch -> j = 3
        fi;
        where[i] = 1 - torch;
        where[j] = 1 - torch;
        torch = 1 - torch;
        now = now + MAX(time[i], time[j])
    od
}

```

Spin יחפש ריצה שתפר את ה-assertion

בחירה אי-דטרמיניסטית לקביעת
חייל שייקח את הלפיד

בחירה אי-דטרמיניסטית לקביעת
חייל נוסף שיבוא איתו



נעבור להגדרות פורמאליות

נגדיר את קבוצת המקומות בתוכנית באופן רקורסיבי

לאחר מכן, באמצעות כללי גזירה, נגדיר את
המעברים בין המקומות השונים



הגדרה רקורסיבית של המקומות

+



כללי גזירה להגדרת המעברים בין המקומות

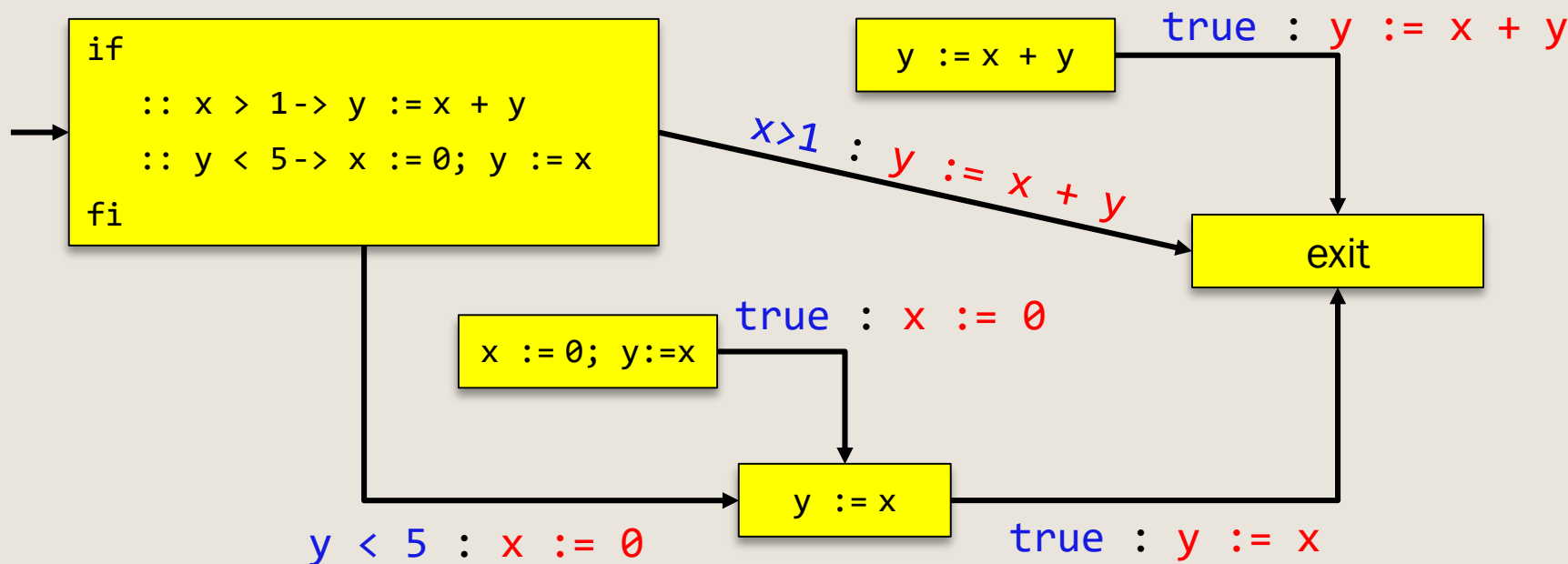
דוגמה: גרף תוכנית למבנה תנאי



```

if
  :: x > 1 -> y := x + y
  :: y < 5 -> x := 0; y := x
fi
  
```

טריק שיעזור להגדיר את הסמנטיקה בנוחות: שם מקום בתוכנית יהיה תוכנית ננו-פרומלה המתארת מה עוד נשאר לעשות כשמגיעים למקום.



דוגמה: גרף תוכנית למבנה תנאי מקוונ



```

if :: x > y ->
  y := 2y;
  if
    :: x > 1 -> y := x + y
    :: true -> x := 0; y := x
  fi
fi

```

```

if :: x > y ->
  y := 2y;
  if
    :: x > 1 -> y := x + y
    :: true -> x := 0; y := x
  fi
fi

```

למען הקריאות, לא ציירנו כאן מקומות שאין אליהם מעבר

$x > y : y := 2y$

```

if
  :: x > 1 -> y := x + y
  :: true -> x := 0; y := x
fi

```

$x > 1 : y := x + y$

exit

$y := x$

$true : x := 0$

$true : y := x$

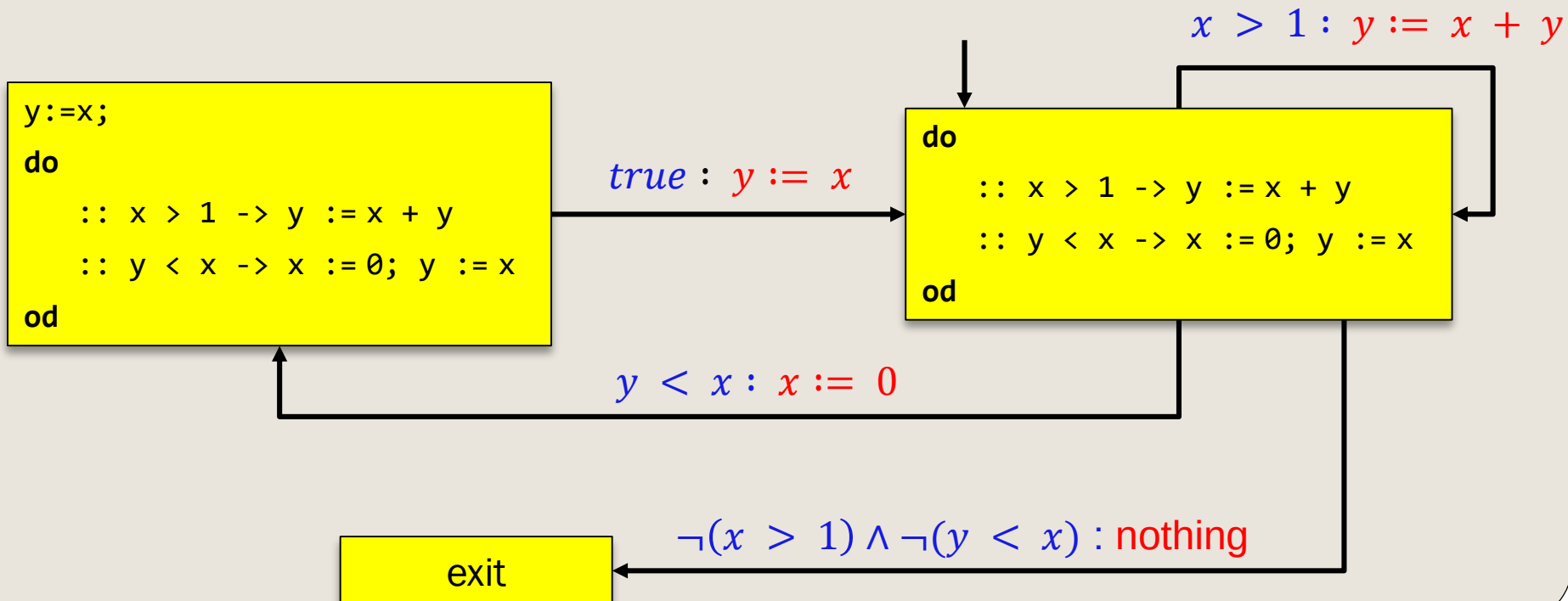
דוגמה: גרף תוכנית למבנה לולאה

```

do
  :: x > 1 -> y := x + y
  :: y < x -> x := 0; y := x
od

```

למען הקריאות, לא ציירנו כאן מקומות שאין אליהם מעבר





הגדרה: תחביר ל-nanoPromela

מערכת ערוצים $[P_1 | \dots | P_n]$ באשר כל תהליך P_i מוגדר ע"י פקודה:

stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |
atomicstmt | skipstmt | stmt ';' stmt ; ← שרשור פקודות

ifstmt	: 'if' option+ 'fi' ;	פקודות תנאי ולולאות עם אחד או יותר פקודות מותנות שיבחרו באופן לא דטרמיניסטי
dostmt	: 'do' option+ 'od' ;	
assstmt	: VARNAME ':=' intexpr ;	השמות למשתנים
chanreadstmt	: CHANNAME '?' VARNAME ;	קריאות וכתיבות לתורים
chanwritestmt	: CHANNAME '!' intexpr ;	
skipstmt	: 'skip' ;	

atomicstmt :
'atomic {' (VARNAME ':=' intexpr ';')* VARNAME ':=' intexpr '}';

option: '::' boolexpr '->' stmt ;

עץ הפריסה

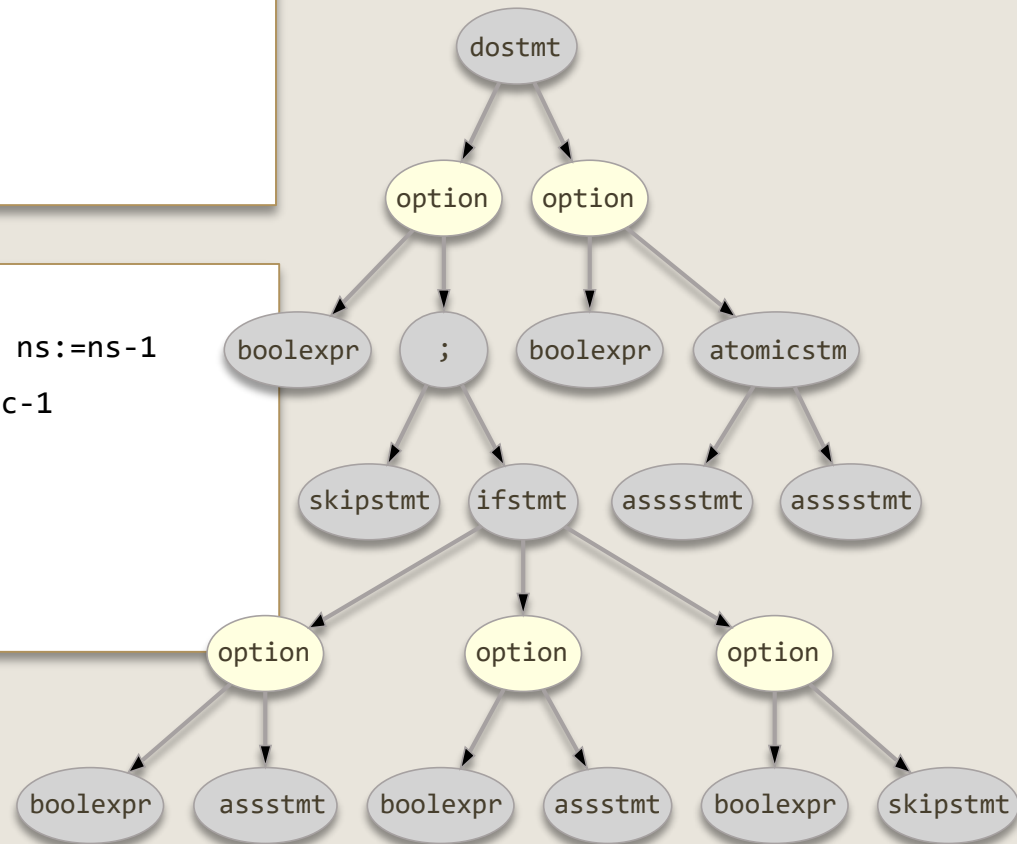
```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' '=' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```

```
do  
  :: true -> skip; if :: ns > 0          -> ns:=ns-1  
      :: nc > 0          -> nc:=nc-1  
      :: nc==0 && ns==0 -> skip  
  fi  
  :: true -> atomic {nc:=10; ns:=10}  
od
```





עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' '=' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```

```
do  
  :: true -> skip;  
      if :: ns > 0          -> ns:=ns-1  
          :: nc > 0          -> nc:=nc-1  
          :: nc==0 && ns==0 -> skip  
      fi  
  :: true -> atomic {nc:=10; ns:=10}  
od
```



עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' '=' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```

dostmt

```
graph TD; dostmt((dostmt)) --> box1[:: true -> skip; if :: ns > 0 -> ns:=ns-1  
:: nc > 0 -> nc:=nc-1  
:: nc==0 && ns==0 -> skip  
fi]; dostmt --> box2[:: true -> atomic {nc:=10; ns:=10}];
```

```
:: true -> skip;  
if  :: ns > 0      -> ns:=ns-1  
    :: nc > 0      -> nc:=nc-1  
    :: nc==0 && ns==0 -> skip  
fi
```

```
:: true -> atomic {nc:=10; ns:=10}
```



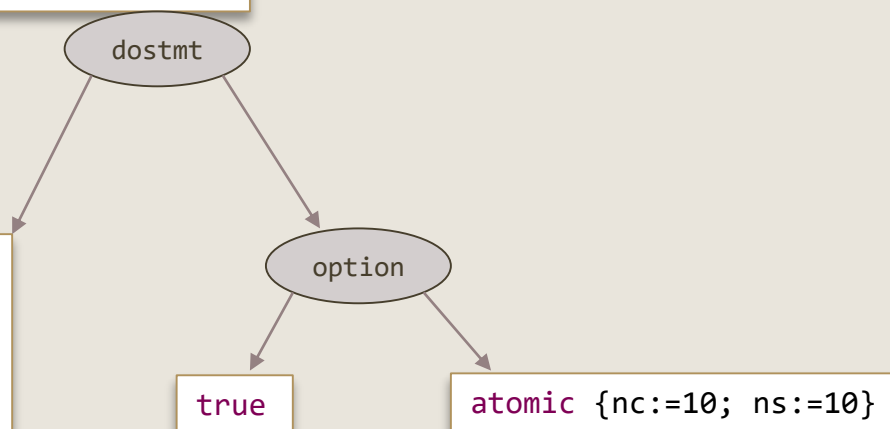
עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':= ' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```



```
::: true -> skip;  
if :: ns > 0      -> ns:=ns-1  
   :: nc > 0      -> nc:=nc-1  
   :: nc==0 && ns==0 -> skip  
fi
```



עץ הפריסה

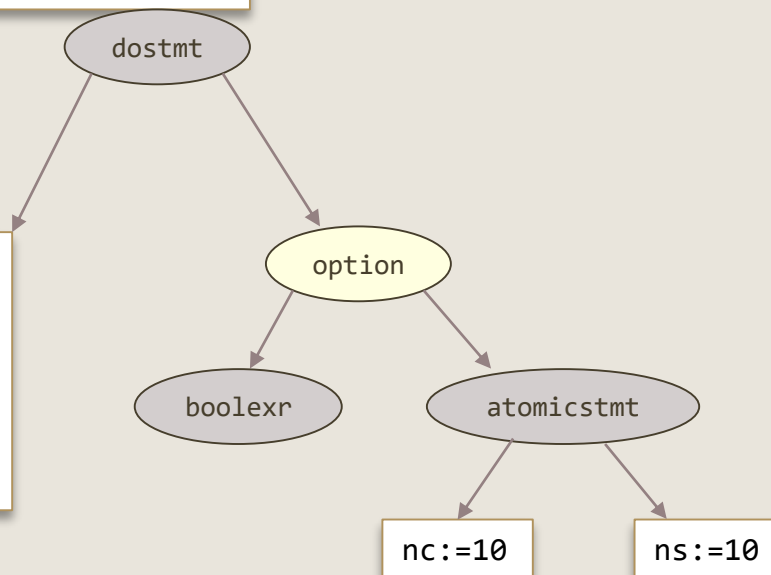
```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' '=' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```

```
:: true -> skip;  
if  :: ns > 0      -> ns:=ns-1  
    :: nc > 0      -> nc:=nc-1  
    :: nc==0 && ns==0 -> skip  
fi
```





עץ הפריסה

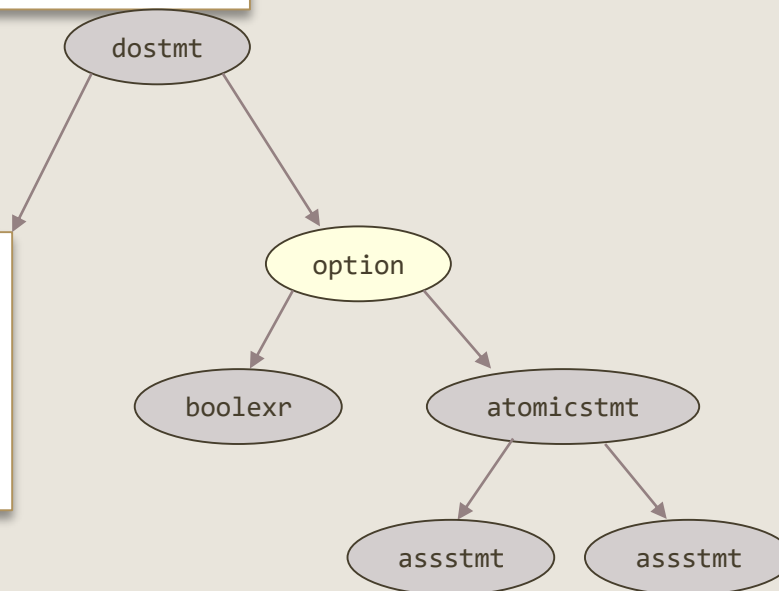
```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':= ' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic { assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```

```
:: true -> skip;  
if  :: ns > 0      -> ns:=ns-1  
    :: nc > 0      -> nc:=nc-1  
    :: nc==0 && ns==0 -> skip  
fi
```





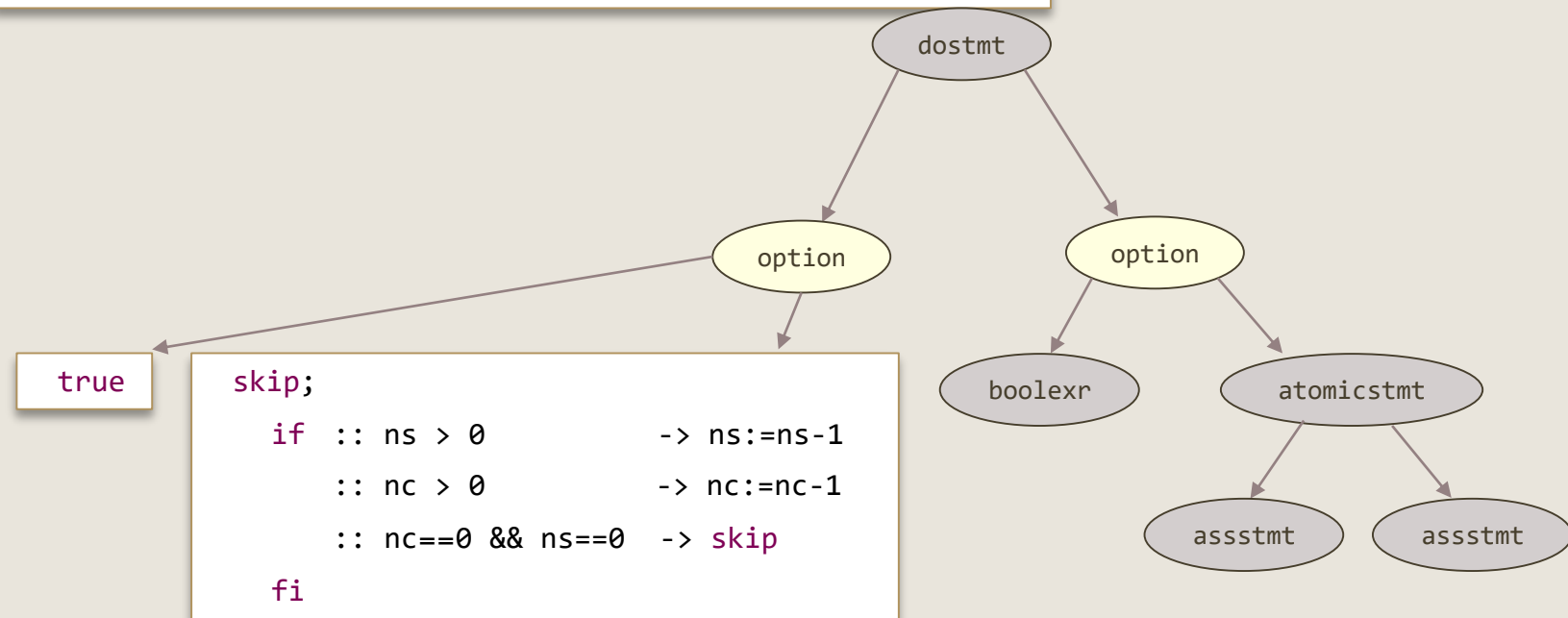
עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':= ' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic { assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```





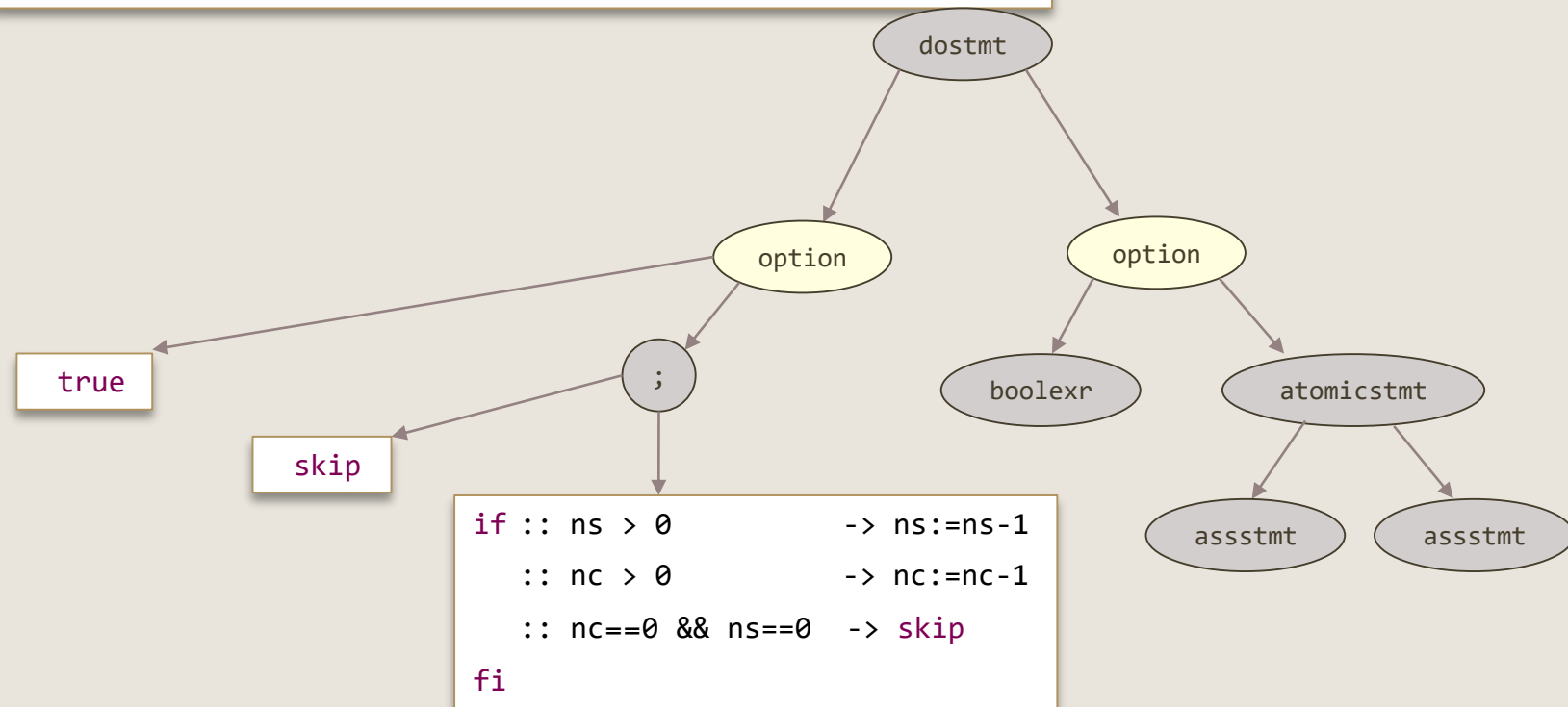
עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':= ' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic { assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```





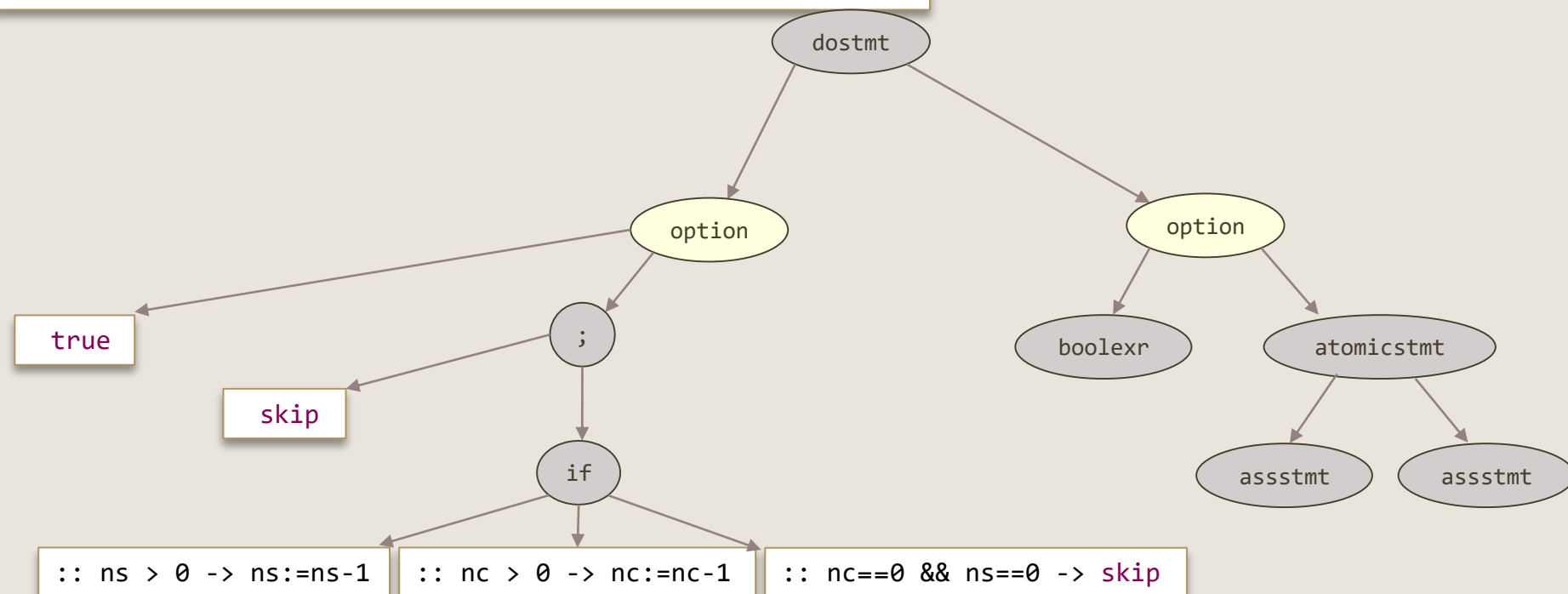
עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic { assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```





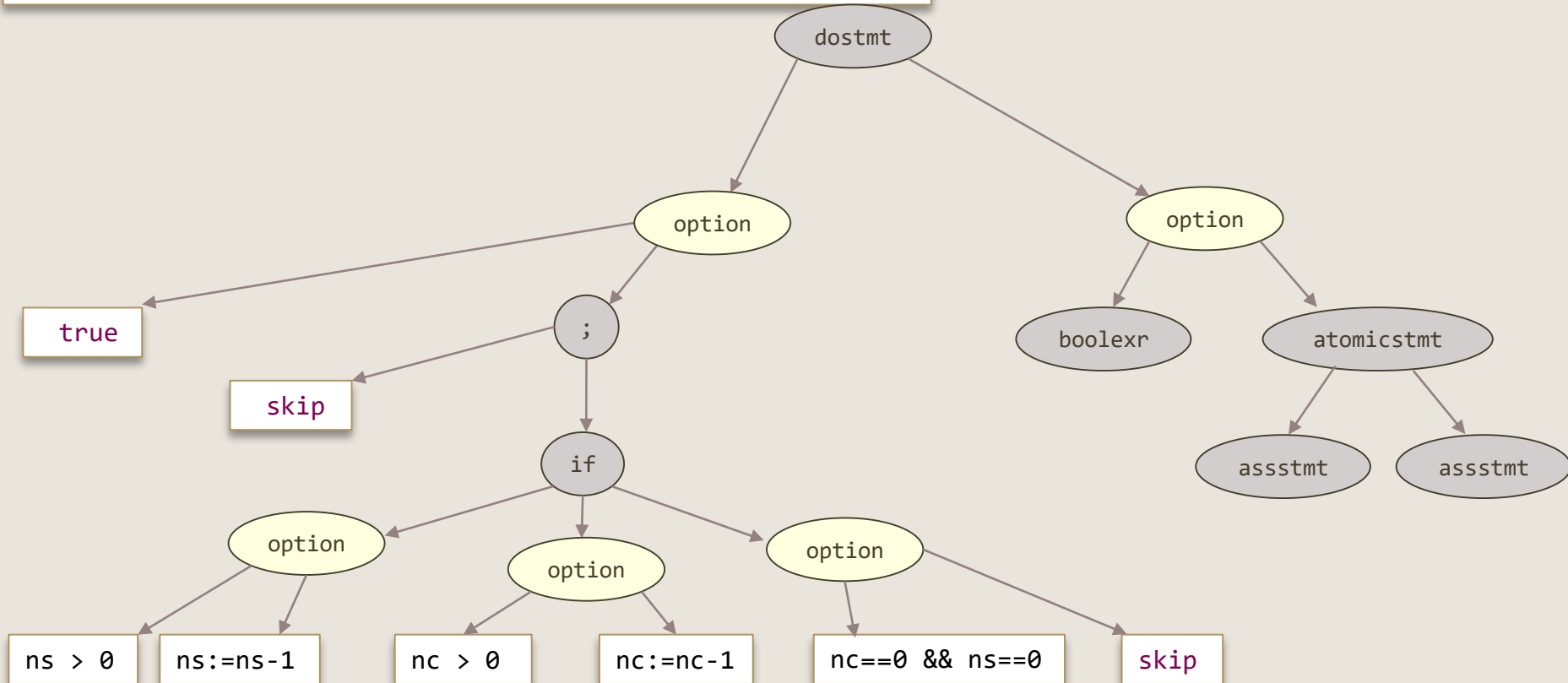
עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {assstmt ';' }* assstmt '};
```

```
option: '::' boolexpr '->' stmt ;
```





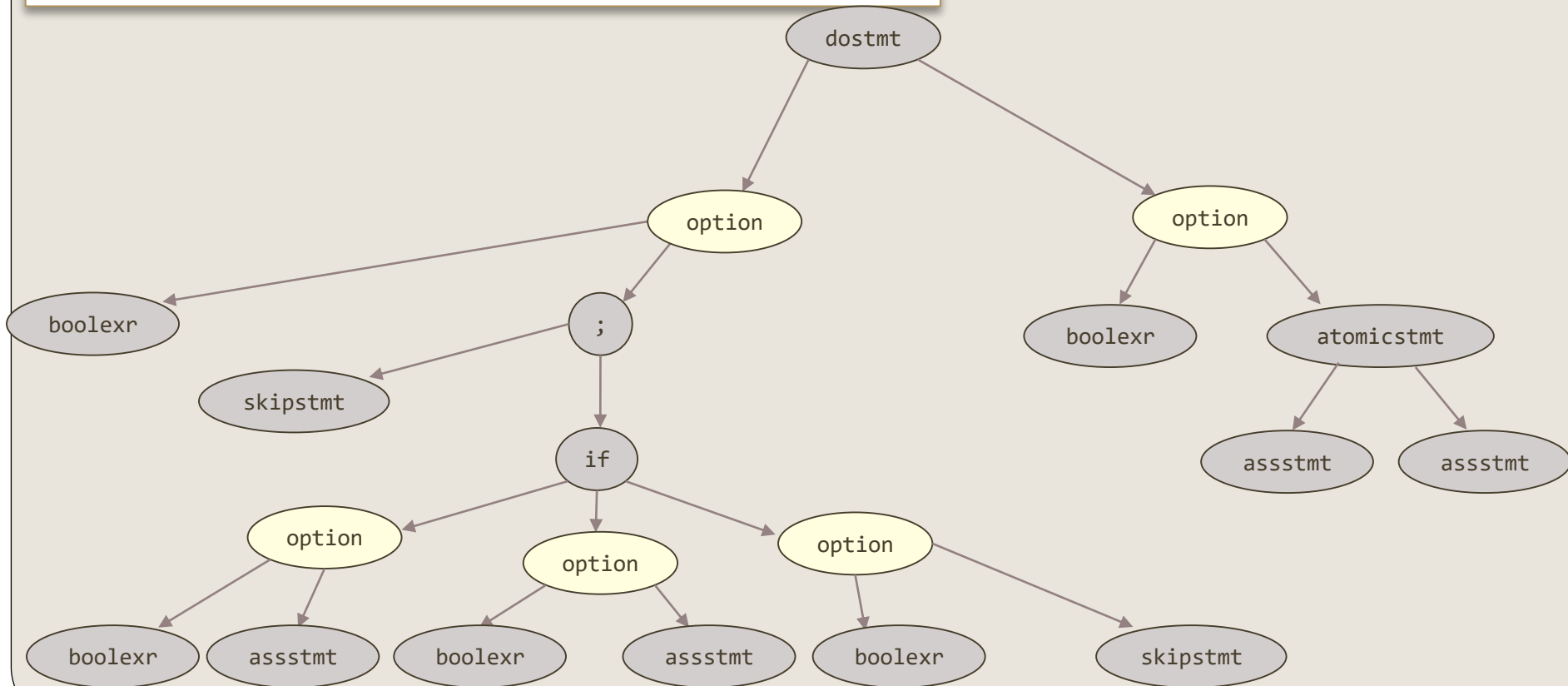
עץ הפריסה

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {assstmt ';' }* assstmt '}' ;
```

```
option: '::' boolexpr '->' stmt ;
```





הגדרה: תחביר ל-nanoPromela

מערכת ערוצים $[P_1 | \dots | P_n]$ באשר כל תהליך P_i מוגדר ע"י פקודה:

stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |
atomicstmt | skipstmt | stmt ';' stmt ; ← שרשור פקודות

ifstmt	: 'if' option+ 'fi' ;	פקודות תנאי ולולאות עם אחד או יותר פקודות מותנות שיבחרו באופן לא דטרמיניסטי
dostmt	: 'do' option+ 'od' ;	
assstmt	: VARNAME ':=' intexpr ;	השמות למשתנים
chanreadstmt	: CHANNAME '?' VARNAME ;	קריאות וכתיבות לתורים
chanwritestmt	: CHANNAME '!' intexpr ;	
skipstmt	: 'skip' ;	

atomicstmt :
'atomic {' (VARNAME ':=' intexpr ';')* VARNAME ':=' intexpr '}' ;

option: '::' boolexpr '->' stmt ;



הגדרה: מקומות = תת הפקודות

מגדירים את המקומות בגרף התוכנית **פְּתַת** הפקודות בתוכנית:

$$Loc = \text{sub}(\text{program})$$

מקרי הבסיס של ההגדרה הִרְקוּרְסִיבִית:

אם

$$\text{stmt} \in \{\text{skip}, x := \text{expr}, c?x, c!\text{expr}, \text{atomic}\{\text{assgn}\}\}$$

מגדירים את תת הפקודות להיות:

$$\text{sub}(\text{stmt}) := \{\text{stmt}, \text{exit}\}$$

המשך ההגדרה, **באופן רִקוּרְסִיבִי**, בשקפים הבאים...

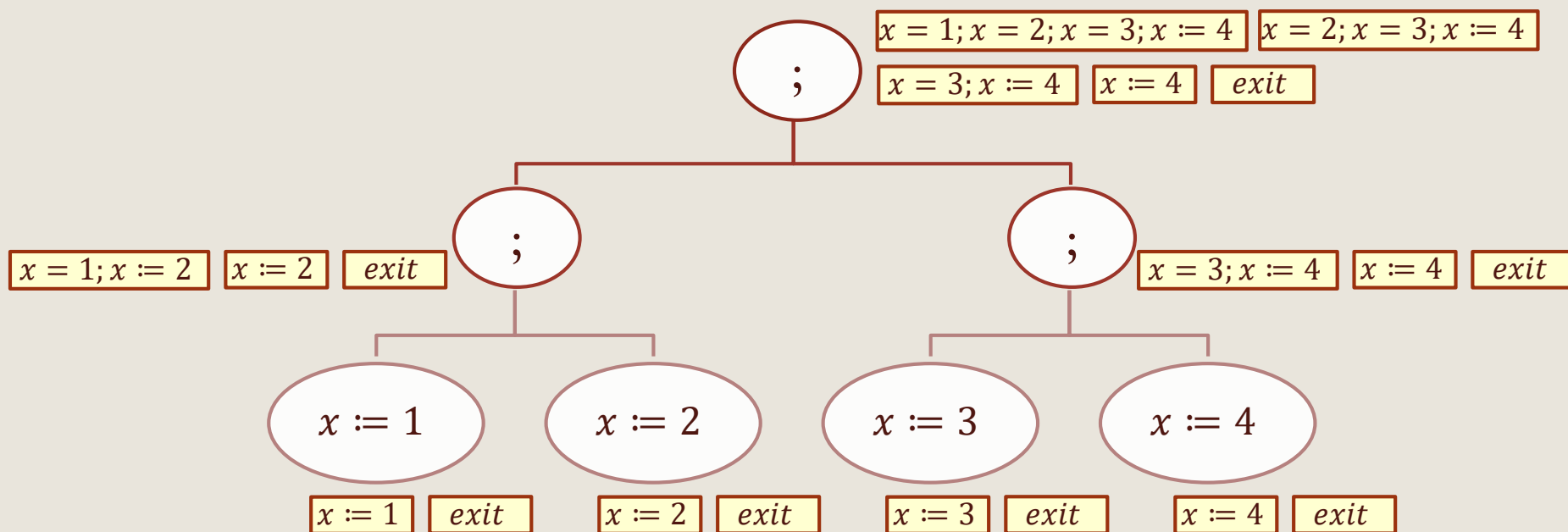
תת-פקודות של פקודות מורכבות

הגדרות רקורסיביות



$$\text{sub}(\text{stmt}_1; \text{stmt}_2) := \{ \text{stmt}'; \text{stmt}_2 : \text{stmt}' \in \text{sub}(\text{stmt}_1) \setminus \{\text{exit}\} \} \cup \text{sub}(\text{stmt}_2)$$

$x = 1; x = 2; x = 3; x := 4$



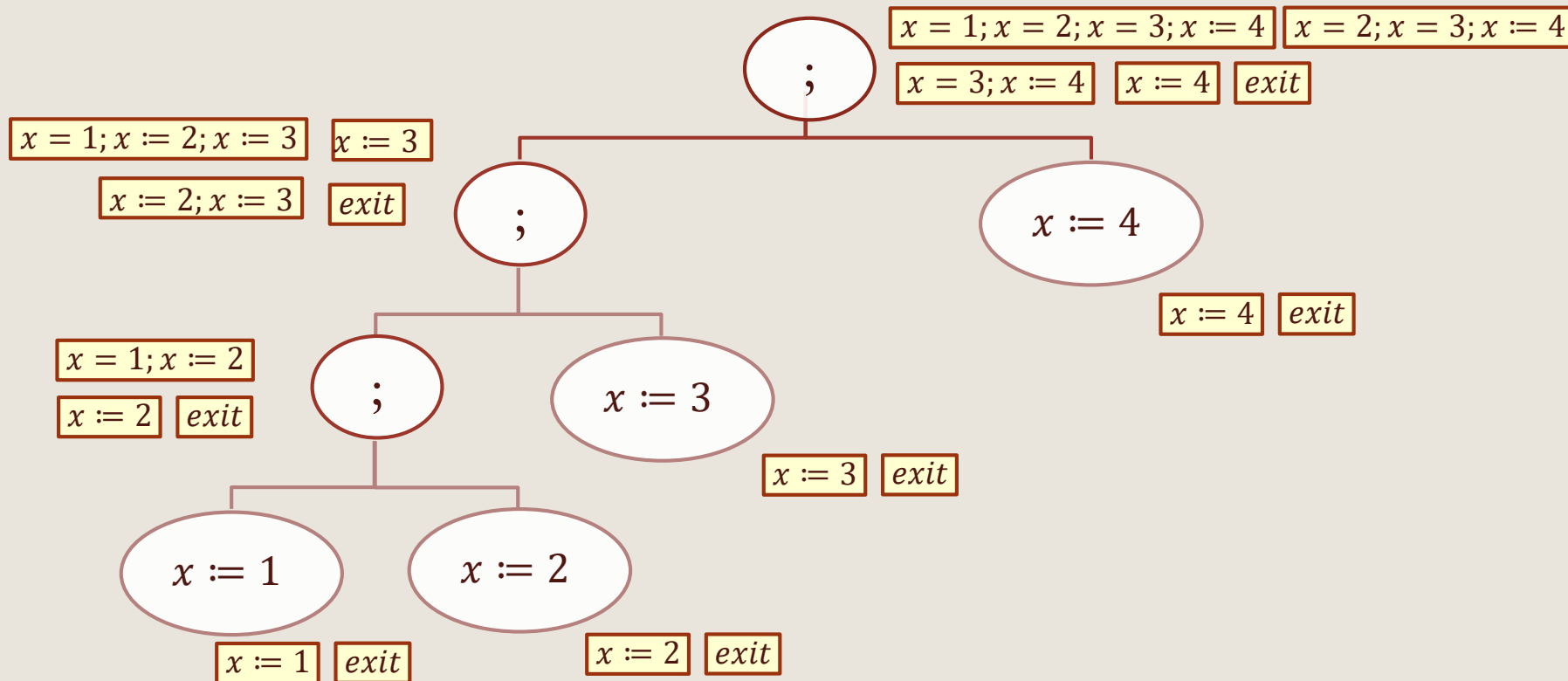
תת-פקודות של פקודות מורכבות

הגדרות רקורסיביות



$$\text{sub}(\text{stmt}_1; \text{stmt}_2) := \{ \text{stmt}'; \text{stmt}_2 : \text{stmt}' \in \text{sub}(\text{stmt}_1) \setminus \{\text{exit}\} \} \cup \text{sub}(\text{stmt}_2)$$

$x = 1; x = 2; x = 3; x := 4$



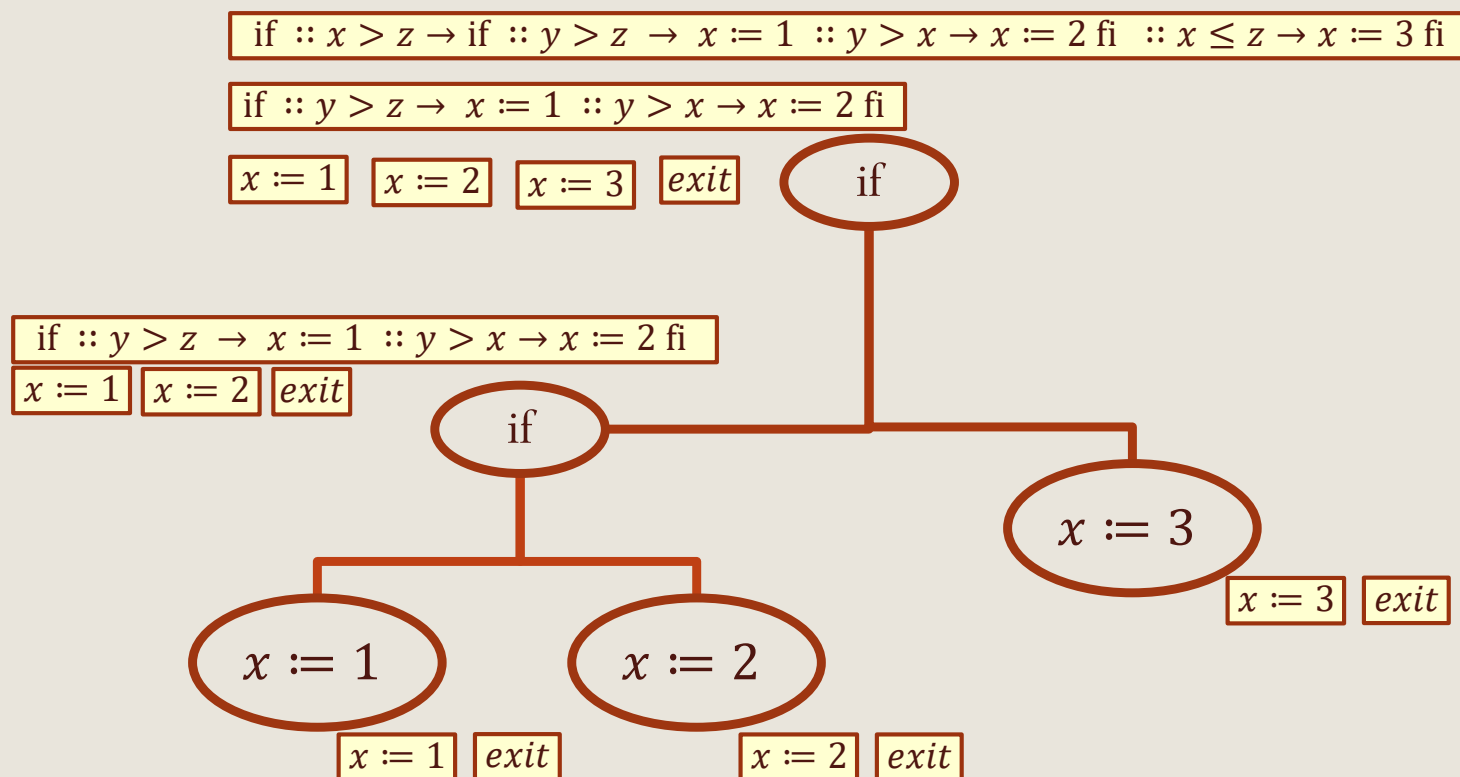
תת-פקודות של פקודות מורכבות

הגדרות רקורסיביות

$$sub \left(\overbrace{\text{if} \ :: g_1 \rightarrow stmt_1 \ \dots \ :: g_n \rightarrow stmt_n}^{\text{cond_cmd}} \text{fi} \right) :=$$

$$\{\text{cond_cmd}, \text{exit}\} \cup sub(stmt_1) \cup \dots \cup sub(stmt_n)$$

if
 $\ :: x > z \rightarrow$ if
 $\ \ \ \ \ \ :: y > z \rightarrow x := 1$
 $\ \ \ \ \ \ :: y > x \rightarrow x := 2$
 $\ \ \ \ \ \text{fi}$
 $\ :: x \leq z \rightarrow x := 3$
 fi



תת-פקודות של פקודות מורכבות

הגדרות רקורסיביות



$$\text{sub}(\text{stmt}_1; \text{stmt}_2) := \\ \{ \text{stmt}' ; \text{stmt}_2 : \text{stmt}' \in \text{sub}(\text{stmt}_1) \setminus \{\text{exit}\} \} \cup \text{sub}(\text{stmt}_2)$$

$$\text{sub} \left(\overbrace{\text{if} :: g_1 \rightarrow \text{stmt}_1 \cdots :: g_n \rightarrow \text{stmt}_n \text{ fi}}^{\text{cond_cmd}} \right) := \\ \{ \text{cond_cmd}, \text{exit} \} \cup \text{sub}(\text{stmt}_1) \cup \cdots \cup \text{sub}(\text{stmt}_n)$$

$$\text{sub} \left(\overbrace{\text{do} :: g_1 \rightarrow \text{stmt}_1 \cdots :: g_n \rightarrow \text{stmt}_n \text{ od}}^{\text{loop_cmd}} \right) := \\ \{ \text{loop_cmd}, \text{exit} \} \cup \bigcup_{i=1, \dots, n} \{ \text{stmt}' ; \text{loop_cmd} : \text{stmt}' \in \text{sub}(\text{stmt}_i) \setminus \{\text{exit}\} \}$$

תת-פקודות של פקודות מורכבות

הגדרות רקורסיביות



$$\text{sub} \left(\overbrace{\text{do} :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n}^{\text{loop_cmd}} \text{od} \right) :=$$
$$\{\text{loop_cmd}, \text{exit}\} \cup \bigcup_{i=1, \dots, n} \{ stmt'; \text{loop_cmd} : stmt' \in \text{sub}(stmt_i) \setminus \{\text{exit}\} \}$$

תת-פקודות של פקודות מורכבות

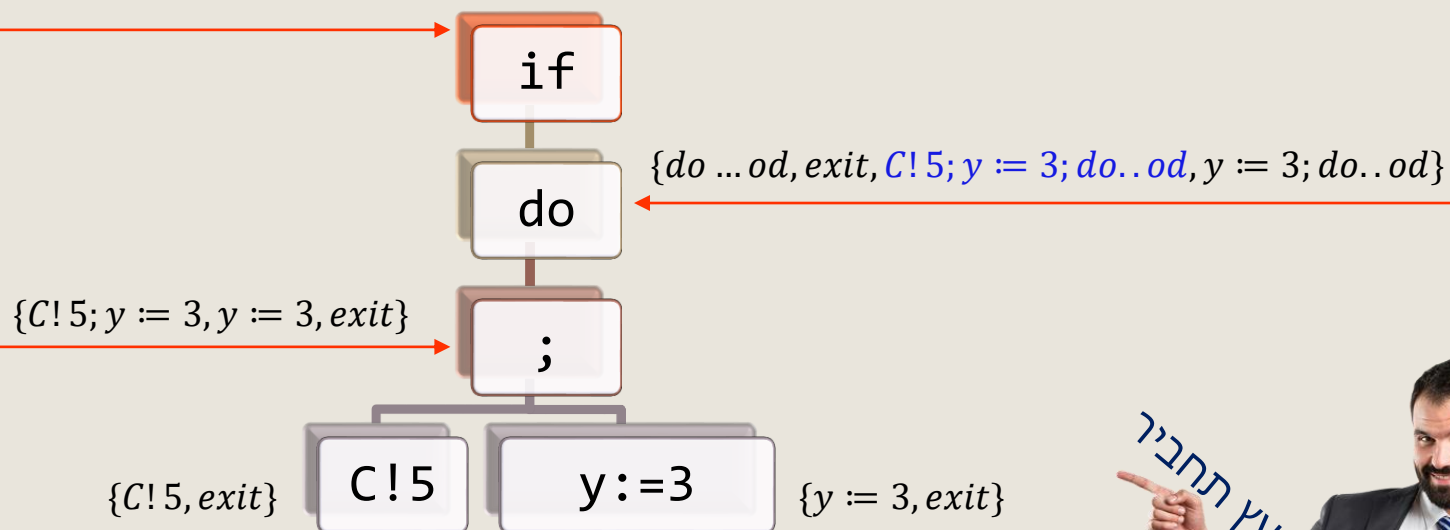
הגדרות רקורסיביות

$$\text{sub}(\text{stmt}_1; \text{stmt}_2) := \{ \text{stmt}' ; \text{stmt}_2 : \text{stmt}' \in \text{sub}(\text{stmt}_1) \setminus \{\text{exit}\} \} \cup \text{sub}(\text{stmt}_2)$$

$$\text{sub} \left(\overbrace{\text{if} :: g_1 \rightarrow \text{stmt}_1 \cdots :: g_n \rightarrow \text{stmt}_n \text{ fi}}^{\text{cond_cmd}} \right) := \{\text{cond_cmd}, \text{exit}\} \cup \text{sub}(\text{stmt}_1) \cup \cdots \cup \text{sub}(\text{stmt}_n)$$

$$\text{sub} \left(\overbrace{\text{do} :: g_1 \rightarrow \text{stmt}_1 \cdots :: g_n \rightarrow \text{stmt}_n \text{ od}}^{\text{loop_cmd}} \right) := \{\text{loop_cmd}, \text{exit}\} \cup \bigcup_{i=1, \dots, n} \{ \text{stmt}' ; \text{loop_cmd} : \text{stmt}' \in \text{sub}(\text{stmt}_i) \setminus \{\text{exit}\} \}$$

$\{\text{if} \dots \text{fi}, \text{exit}, \text{do} \dots \text{od}, C!5; y := 3; \text{do} \dots \text{od}, y := 3; \text{do} \dots \text{od}\}$



if :: x < 2 -> do :: z > 3 -> C!5; y := 3 od fi

עץ תחביר

תת-פקודות של פקודות מורכבות

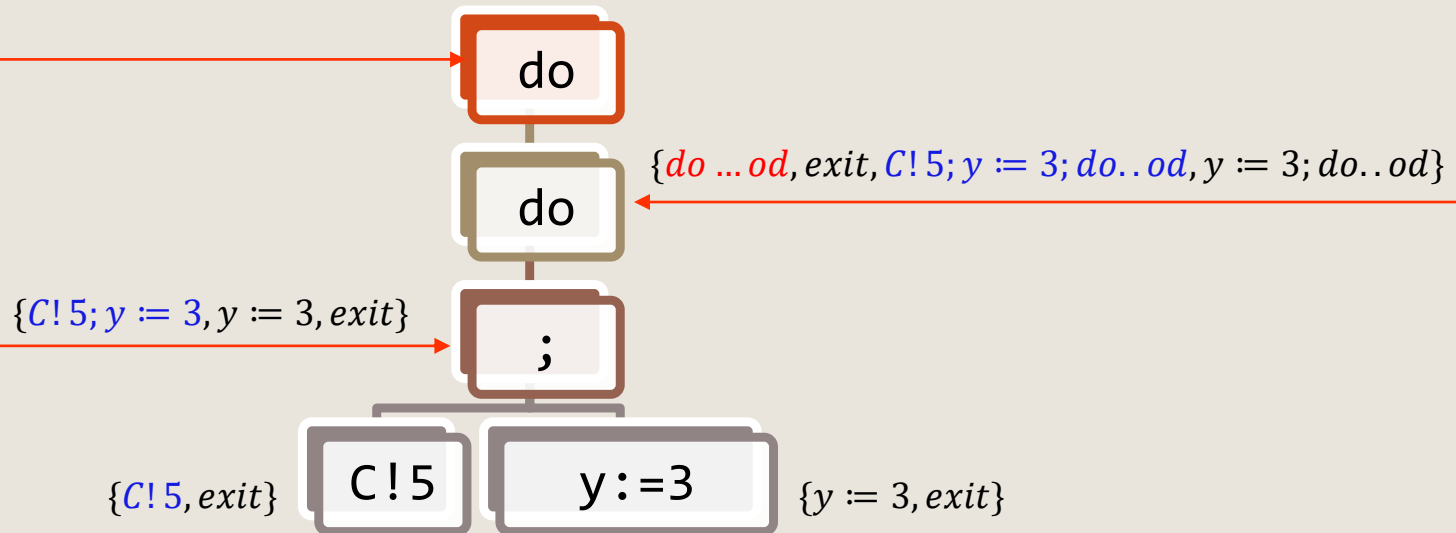
הגדרות רקורסיביות

$$\text{sub}(\text{stmt}_1; \text{stmt}_2) := \{ \text{stmt}' ; \text{stmt}_2 : \text{stmt}' \in \text{sub}(\text{stmt}_1) \setminus \{\text{exit}\} \} \cup \text{sub}(\text{stmt}_2)$$

$$\text{sub} \left(\overbrace{\text{if } g_1 \rightarrow \text{stmt}_1 \cdots \text{if}}^{\text{cond_cmd}} \right) := \{\text{cond_cmd}, \text{exit}\} \cup \text{sub}(\text{stmt}_1) \cup \cdots \cup \text{sub}(\text{stmt}_n)$$

$$\text{sub} \left(\overbrace{\text{do } g_1 \rightarrow \text{stmt}_1 \cdots \text{do}}^{\text{loop_cmd}} \right) := \{\text{loop_cmd}, \text{exit}\} \cup \bigcup_{i=1, \dots, n} \{ \text{stmt}' ; \text{loop_cmd} : \text{stmt}' \in \text{sub}(\text{stmt}_i) \setminus \{\text{exit}\} \}$$

$\{\text{do } \dots \text{od}, \text{exit}, \text{do } \dots \text{od}; \text{do } \dots \text{od}, C!5; y := 3; \text{do } \dots \text{od}; \text{do } \dots \text{od}, y := 3; \text{do } \dots \text{od}; \text{do } \dots \text{od}\}$



do :: x < 2 -> do :: z > 3 -> C!5; y := 3 od od



דוגמה: חישוב *sub* באופן רקורסיבי



Sub = { *y*:=2*y*; *innerif*, *innerif*, *y*:=*x*+*y*,
x:=0; *y*:=*x*, *y*:=*x*, *exit* }

y := 2*y*;
 if
 :: *x* > 1 -> *y* := *x* + *y*
 :: true -> *x* := 0; *y* := *x*
 fi

y := 2*y*

Sub = {*y*:=2*y*, *exit*}

if
 :: *x* > 1 -> *y* := *x* + *y*
 :: true -> *x* := 0; *y* := *x*
 fi
innerif

y := *x* + *y*

Sub = {*y*:=*x*+*y*, *exit*}

x := 0; *y* := *x*

Sub = {*x*:=0; *y*:=*x*, *y*:=*x*, *exit*}

Sub = {*x*:=0, *exit*} *x* := 0

y := *x*

Sub = {*y*:=*x*, *exit*}

if :: *x* > *y* -> *y* := 2*y*;
 if
 :: *x* > 1 -> *y* := *x* + *y*
 :: true -> *x* := 0; *y* := *x*
 fi
 fi
outerif

Sub = { *outerif*, *y*:=2*y*; *innerif*,
innerif, *y*:=*x*+*y*,
x:=0; *y*:=*x*, *y*:=*x*, *exit* }

Sub = { *innerif*, *y*:=*x*+*y*, *x*:=0; *y*:=*x*,
y:=*x*, *exit* }



מגדירים את הקשתות בגרף התוכנית

$$\frac{}{\text{skip} \xrightarrow{\text{true} : \text{nothing}} \text{exit}}$$

0

$$\frac{}{x := \text{expr} \xrightarrow{\text{true} : x := \text{expr}} \text{exit}}$$

$$\frac{}{C?x \xrightarrow{\text{true} : C?x} \text{exit}}$$

$$\frac{}{C!x \xrightarrow{\text{true} : C!x} \text{exit}}$$

$$\frac{}{\text{atomic}\{x_1 := \text{exp}_1; \dots x_m := \text{exp}_m\} \xrightarrow{\text{true} : x_1 := \text{exp}_1; \dots x_m := \text{exp}_m} \text{exit}}$$

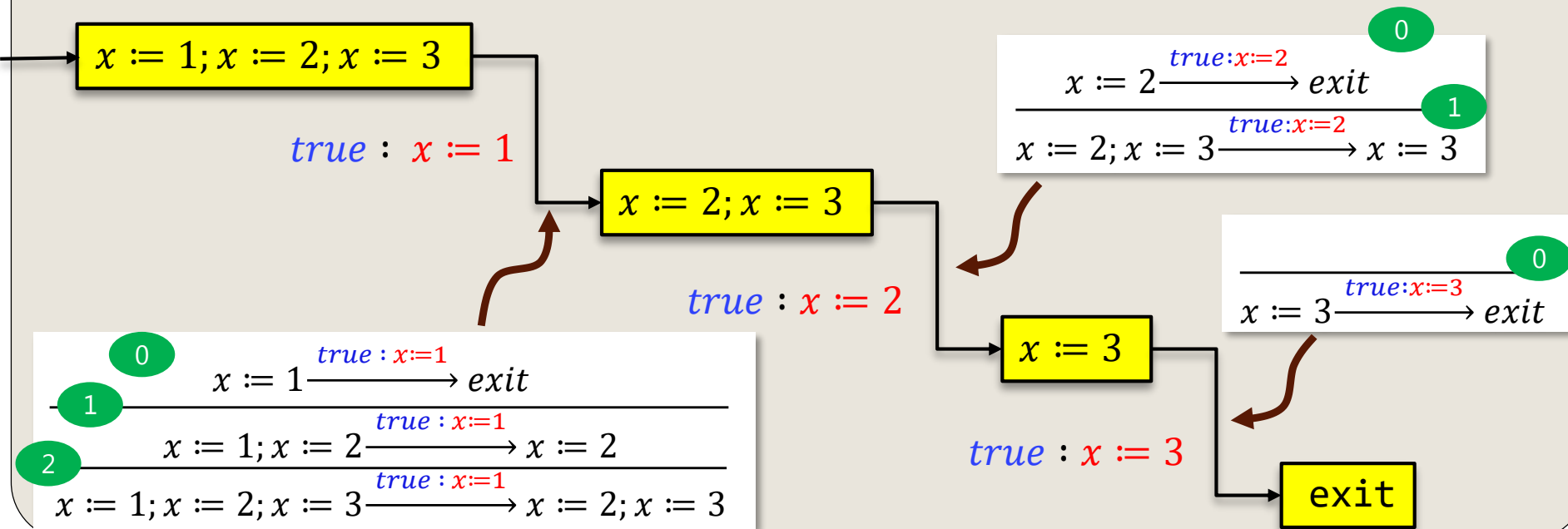
כללי הסק לשרשור

1

$$\frac{stmt_1 \xrightarrow{g:\alpha} stmt'_1 \quad \wedge \quad stmt'_1 \neq \text{exit}}{stmt_1; stmt_2 \xrightarrow{g:\alpha} stmt'_1; stmt_2}$$

2

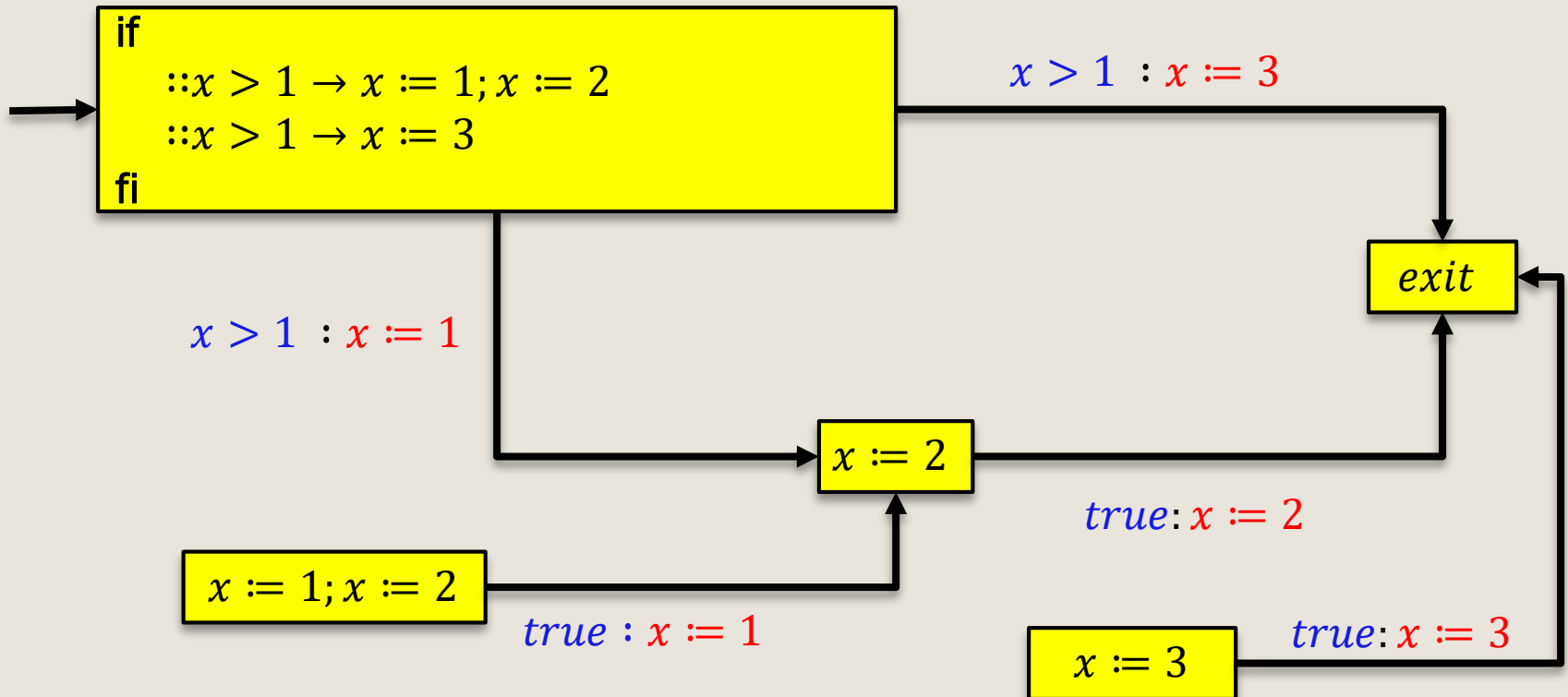
$$\frac{stmt_1 \xrightarrow{g:\alpha} \text{exit}}{stmt_1; stmt_2 \xrightarrow{g:\alpha} stmt_2}$$



כללי הסק לפקודות תנאי

3

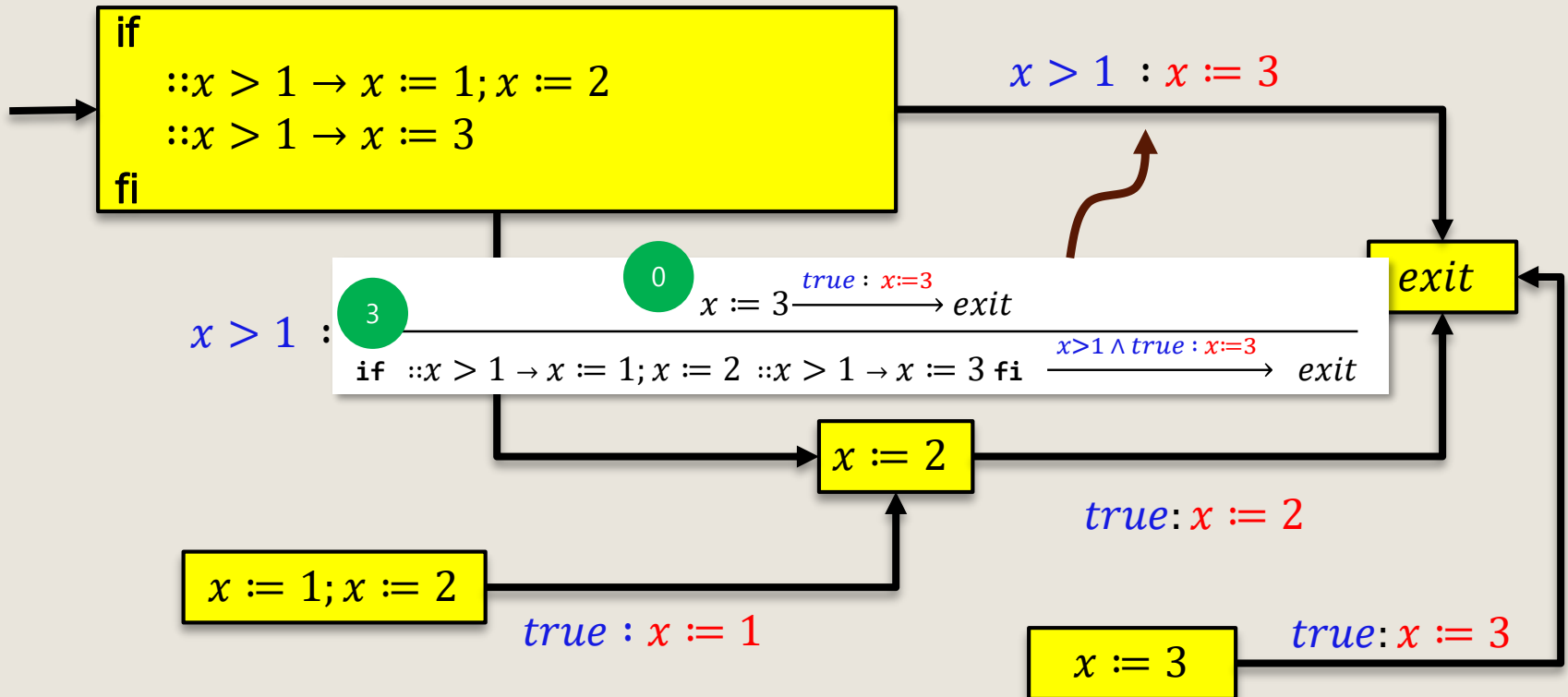
$$\frac{stmt_i \xrightarrow{h:\alpha} stmt'_i}{\text{if } ::g_1 \rightarrow stmt_1 \ \dots \ ::g_n \rightarrow stmt_n \text{ fi} \xrightarrow{g_i \wedge h:\alpha} stmt'_i}$$



כללי הסק לפקודות תנאי

3

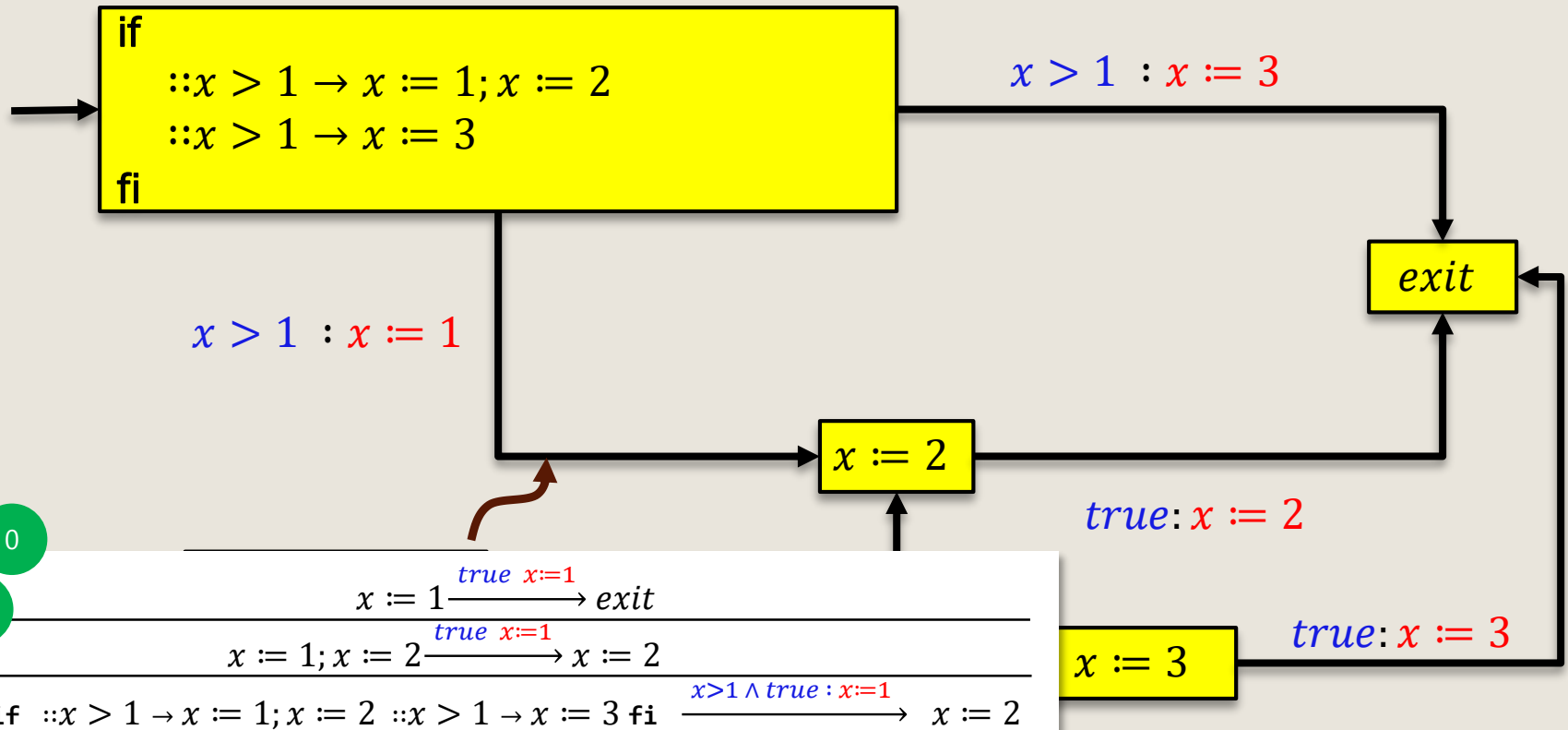
$$\frac{stmt_i \xrightarrow{h:\alpha} stmt'_i}{\text{if } ::g_1 \rightarrow stmt_1 \ \dots \ ::g_n \rightarrow stmt_n \text{ fi} \xrightarrow{g_i \wedge h:\alpha} stmt'_i}$$



כללי הסק לפקודות תנאי

3

$$\frac{stmt_i \xrightarrow{h:\alpha} stmt'_i}{\text{if } ::g_1 \rightarrow stmt_1 \ \dots \ ::g_n \rightarrow stmt_n \text{ fi} \xrightarrow{g_i \wedge h:\alpha} stmt'_i}$$



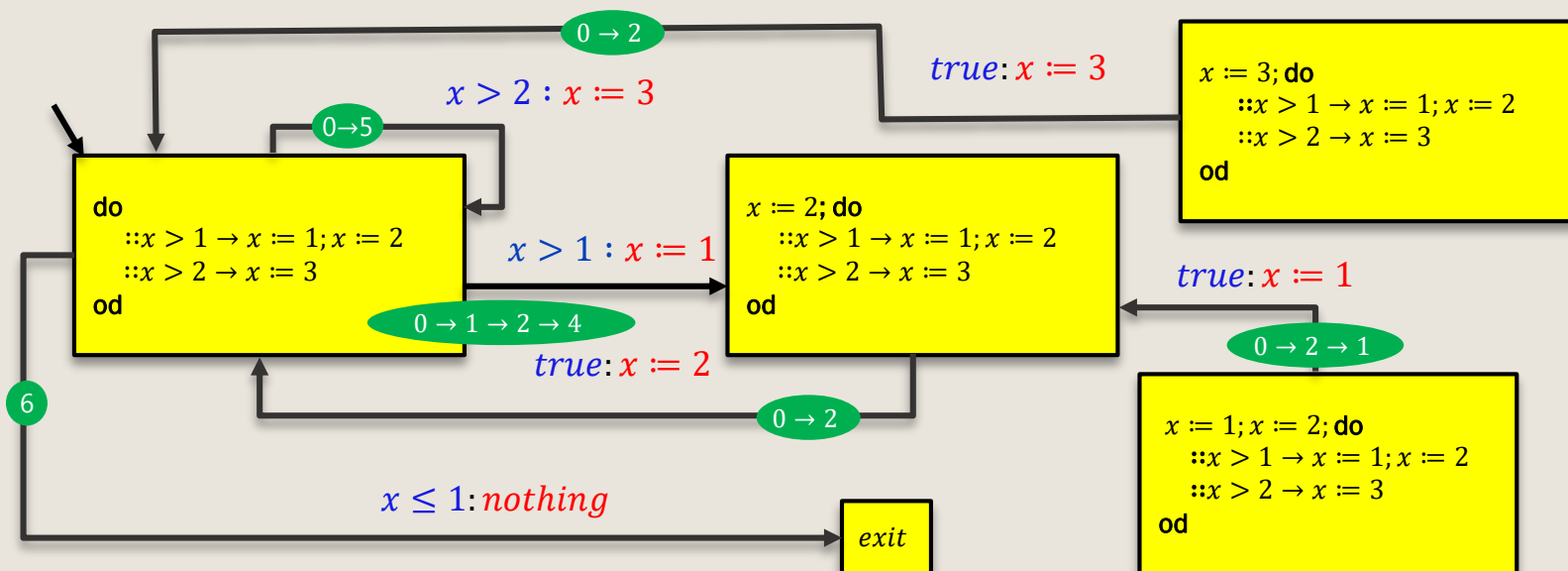
כללי הסק לפקודות לולאה



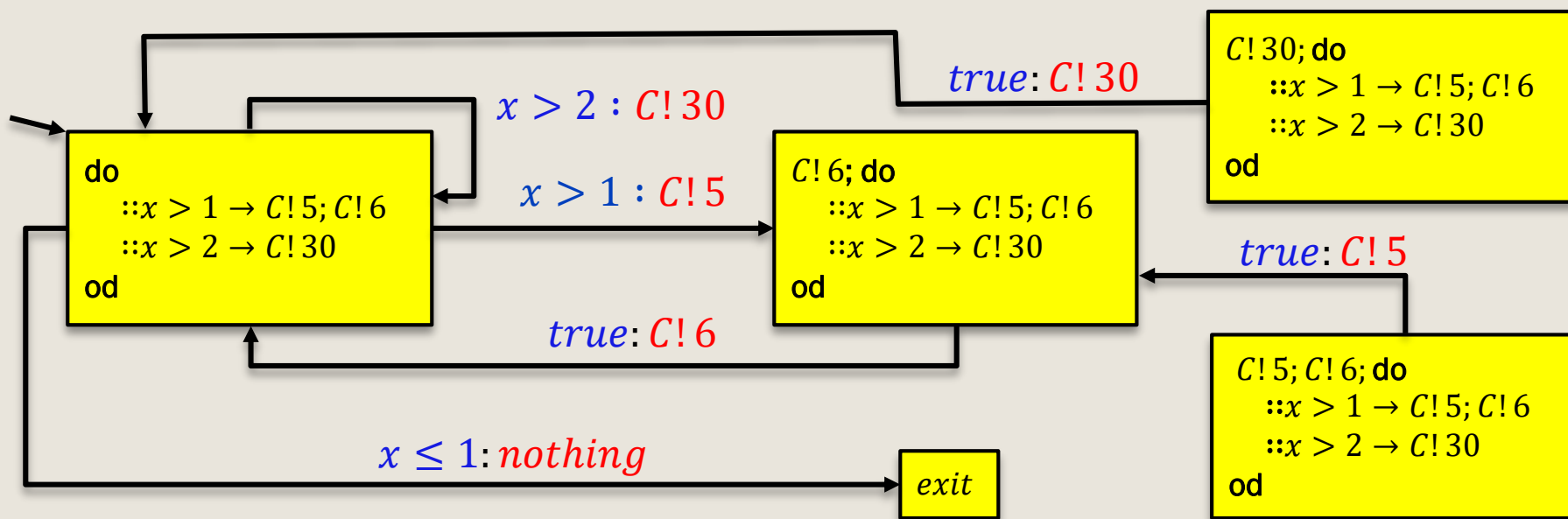
$$\frac{stmt_i \xrightarrow{h:\alpha} stmt'_i \wedge stmt'_i \neq \text{exit}}{do :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n od \xrightarrow{g_i \wedge h:\alpha} stmt'_i; do :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n od} \quad 4$$

$$\frac{stmt_i \xrightarrow{h:\alpha} \text{exit}}{do :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n od \xrightarrow{g_i \wedge h:\alpha} do :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n od} \quad 5$$

$$\frac{}{do :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n od \xrightarrow{\neg g_1 \wedge \cdots \wedge \neg g_n : \text{nothing}} \text{exit}} \quad 6$$



הפקודות הבסיסיות לא משנות הרבה



זה אותו הגרף שראינו בשקף הקודם,
ההבדל היחיד הוא שמות פקודות הבסיס



דוגמה

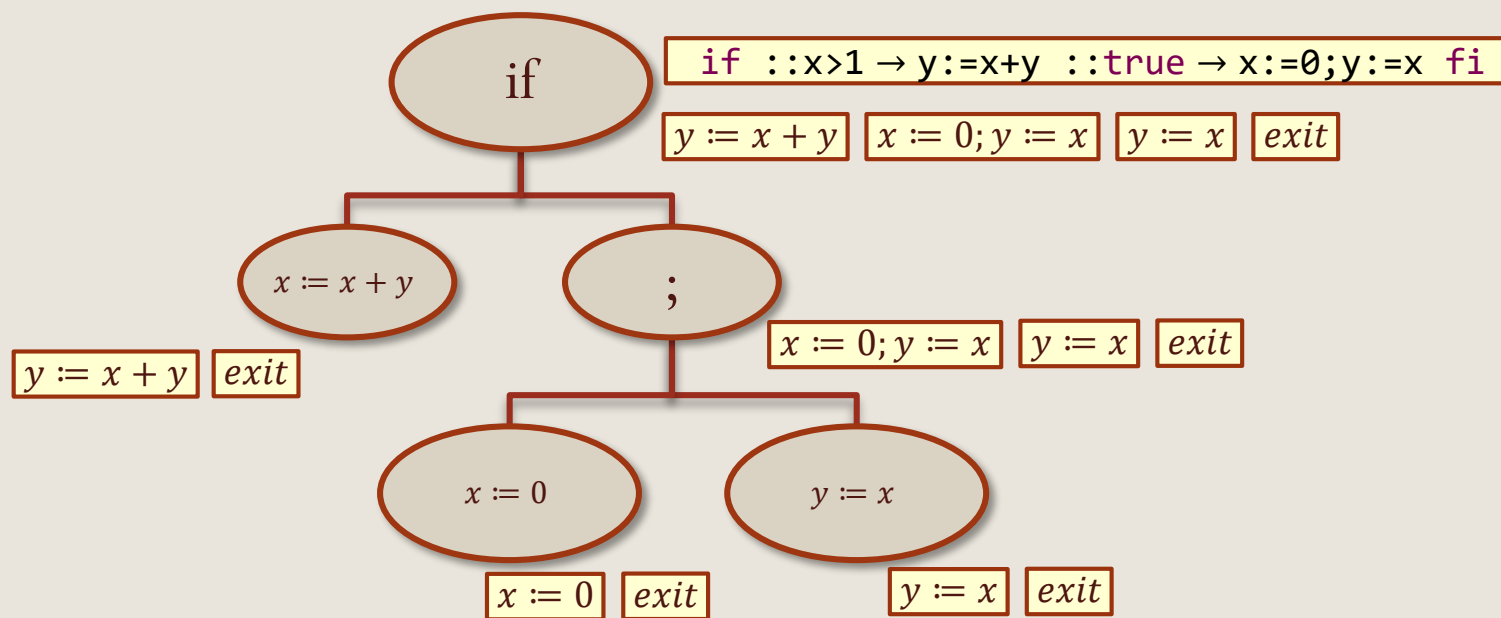
```
if
  :: x > 1 -> y := x + y
  :: true  -> x := 0; y := x
fi
```


דוגמה

```

if
  :: x > 1 -> y := x + y
  :: true -> x := 0; y := x
fi

```



שלב ראשון: מחשבים את המקומות לפי הגדרת *sub*

דוגמה

```
if
  ::  $x > 1 \rightarrow y := x + y$ 
  :: true  $\rightarrow x := 0; y := x$ 
fi
```

if ... fi

$y := x + y$

exit

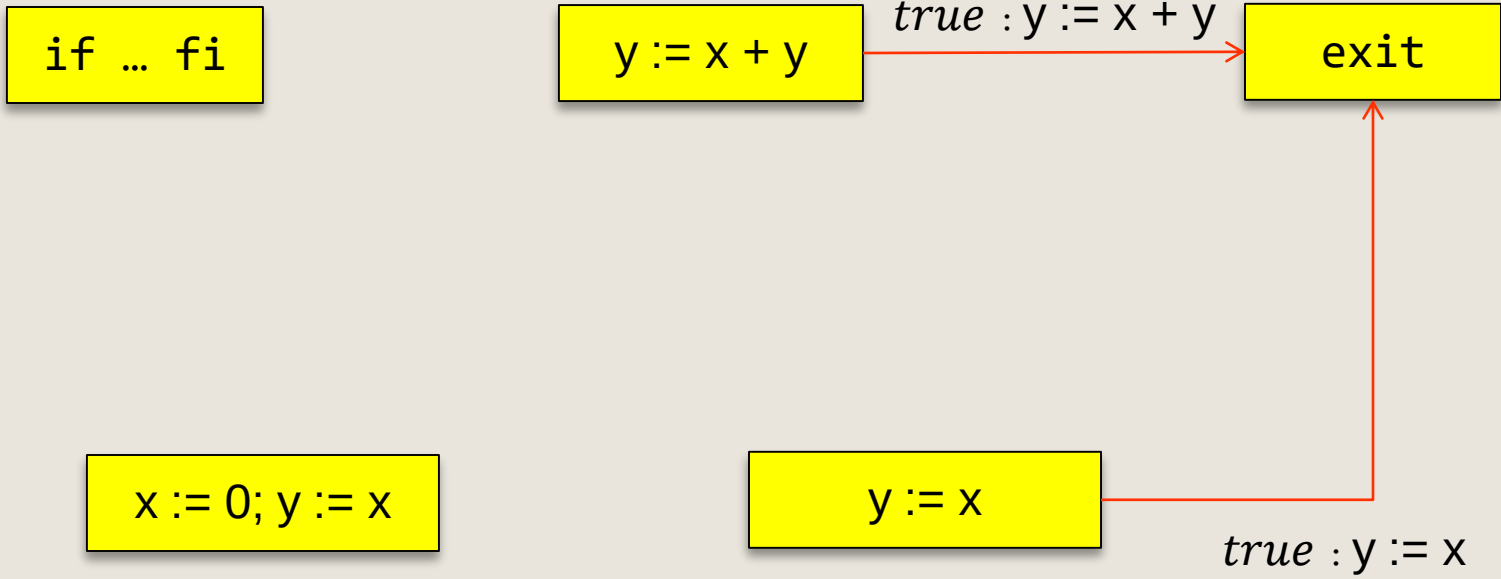
$x := 0; y := x$

$y := x$

שלב שני: גוזרים מעברים לפי הכללים שלמדנו

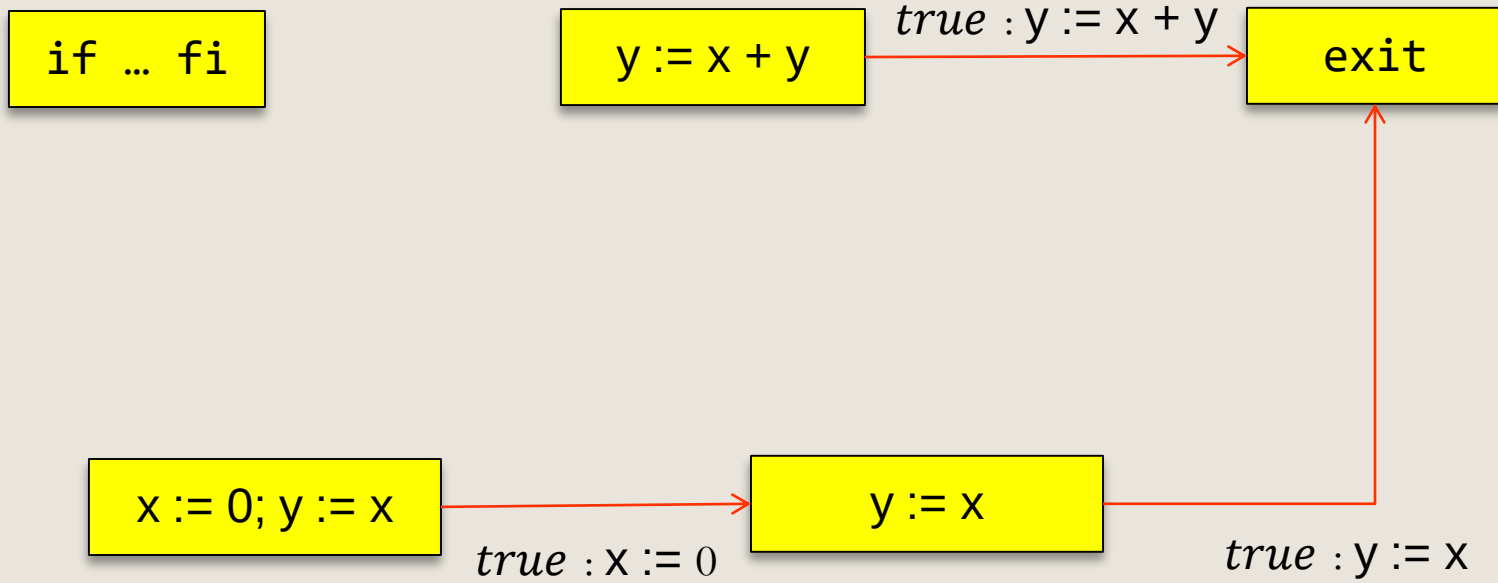
דוגמה: מעברים לפקודות השמה

```
if
  :: x > 1 -> y := x + y
  :: true  -> x := 0; y := x
fi
```



דוגמה: מעבר לפקודת הרכבה

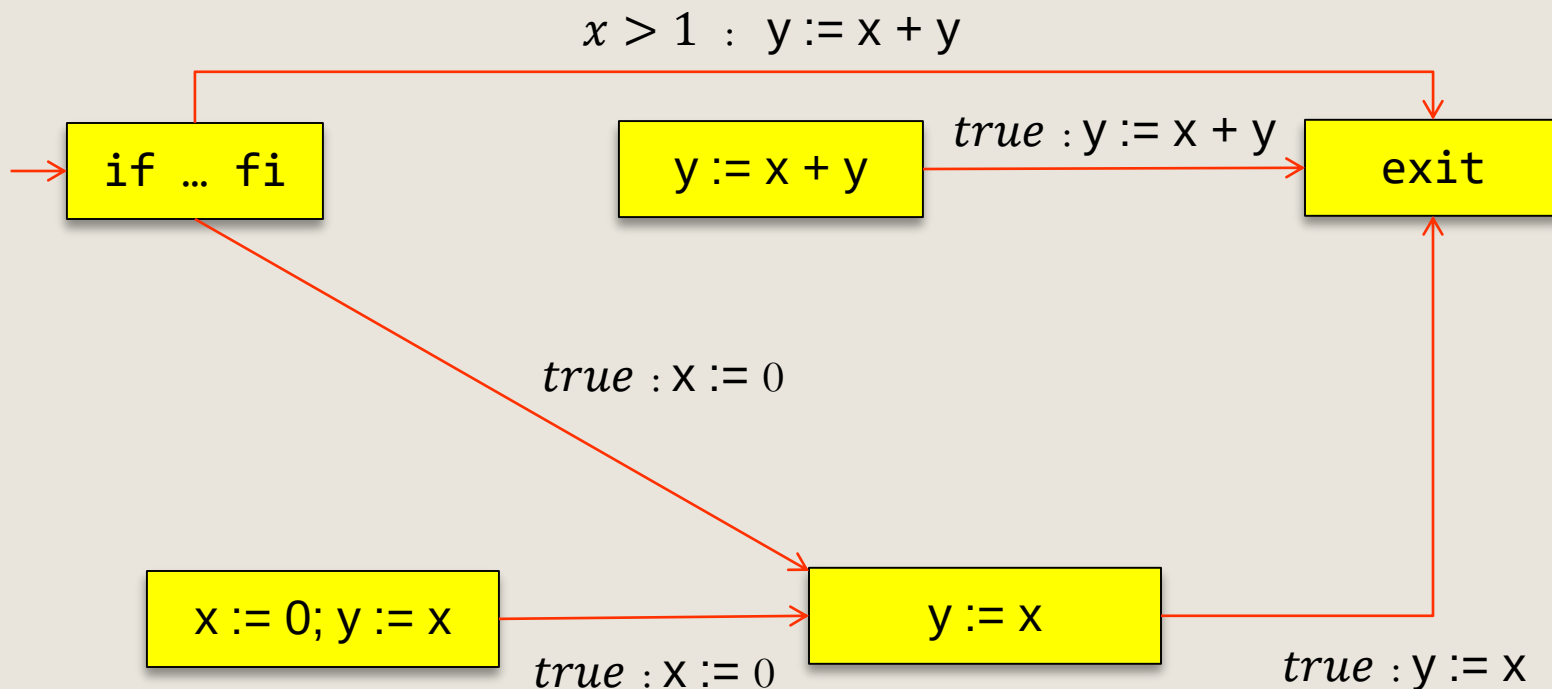
```
if  
  ::  $x > 1 \rightarrow y := x + y$   
  :: true  $\rightarrow x := 0; y := x$   
fi
```





דוגמה: מעברים לפקודת תנאי

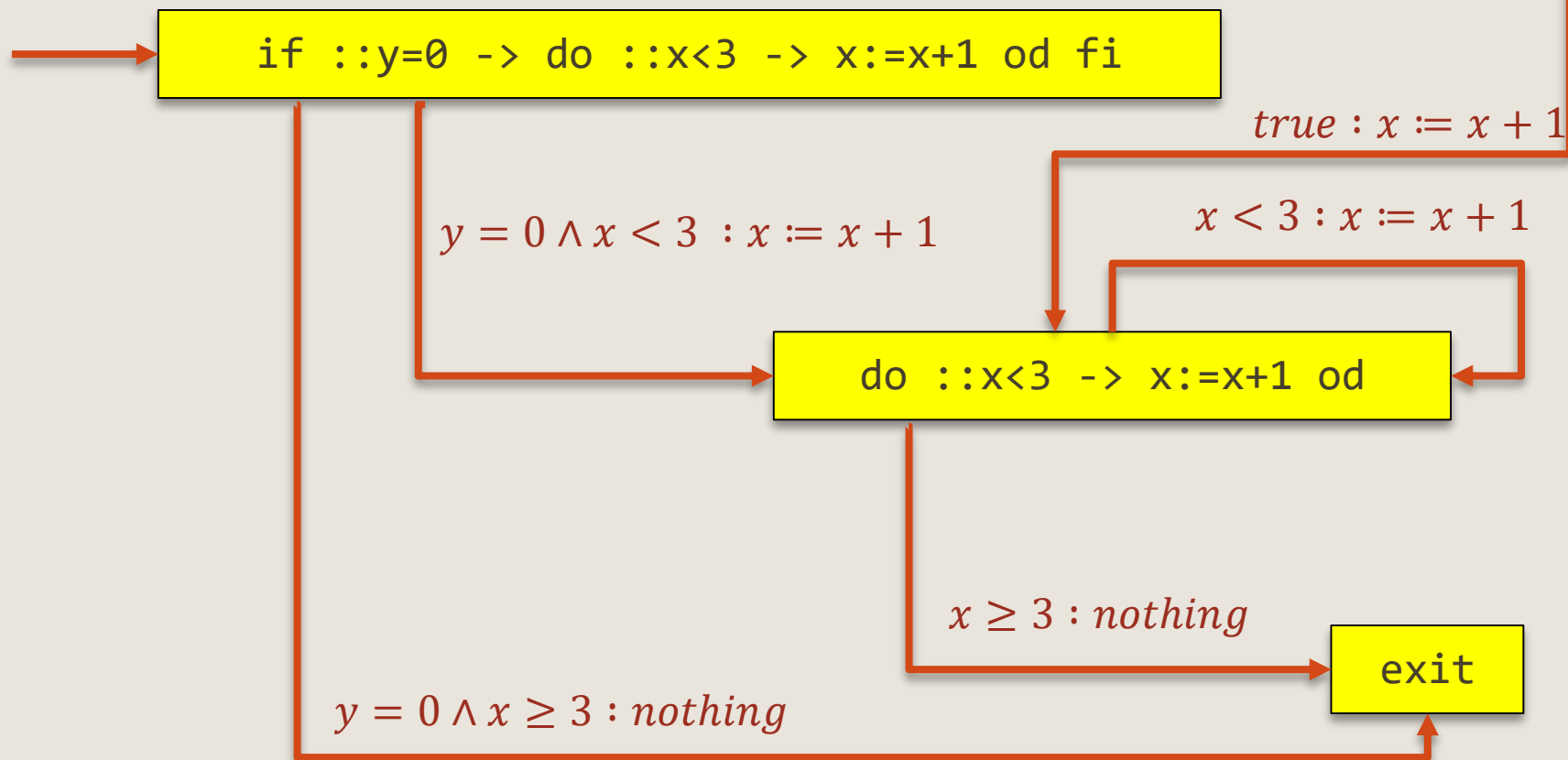
```
if
  ::  $x > 1 \rightarrow y := x + y$ 
  :: true  $\rightarrow x := 0; y := x$ 
fi
```





דוגמה: לפעמים אפשר להגיע גם לפקודה הראשונה בתוך תנאי

```
x:=x+1 ;do ::x<3 -> x:=x+1 od
```





Promela

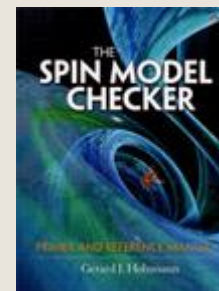
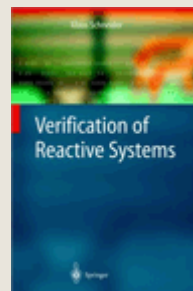
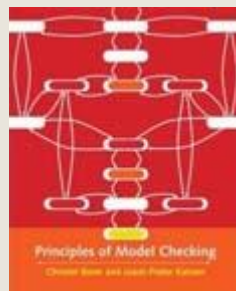
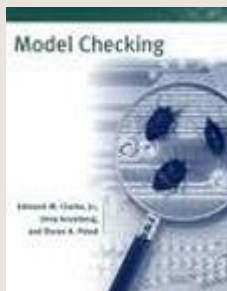
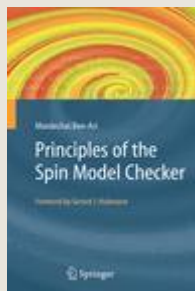
- השפה מכילה מבנים נוספים שלא פרטנו:
- קטעים אטומיים משוכללים – לא רק סדרות של השמות
- מערכים ומבני נתונים נוספים
- יצירה דינאמית של תהליכים
- ועוד...
- הסמנטיקה מוגדרת ע"י תרגום למערכות ערוצים ודרכן למערכות מעברים
- תיאור מלא של התחביר והסמנטיקה המלאים של Promela אפשר למצוא בספרות על SPIN (בשקף הבא)

מקורות נוספים ללימוד SPIN



<http://spinroot.com/spin/whatispin.html>

במעבדות המחלקה מותקנת התוכנה spin שהיא מימוש אלגוריתמי בדיקת המודל עצמם, והתוכנה ispin העוטפת את spin בממשק גרפי המאפשר קביעת פרמטרים באופן נוח ומציג את תוצאות הריצה בדרך שקל יותר להבין.



הבדלים בין Promela ל- nanoPromela



Promela

```
do :: atomic{x > 1 -> y:=x+y}
    :: atomic{y < x -> x:=0}; y:=x
    :: atomic{else -> break}
od
```

nanoPromela

```
do :: x > 1 -> y := x + y
    :: y < x -> x := 0; y := x
od
```